BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HỒ CHÍ MINH
VIỆN ĐÀO TẠO SAU ĐẠI HỌC

———— • ————

TIỂU LUẬN MÔN HỌC
CƠ SỞ DỮ LIỆU

Đề tài:

XÂY DỰNG HỆ THỐNG BÁN HÀNG TRANG SỨC VỚI POSTGRESQL

Giảng viên hướng dẫn: ……
Lớp: ……
Nhóm thực hiện: Nhóm 6
Thành viên: Phùng Anh Tuấn – MSSV: 2541864024
Võ Bùi Duy Thông – MSSV: 2541864019

TP. Hồ Chí Minh, tháng 10 năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TP. Hồ Chí Minh, ngày …… tháng …… năm 2026

Giảng viên

PHÂN CÔNG CÔNG VIỆC NHÓM 6

STT	Thành viên	Nội dung thực hiện	Mức độ
1	Phùng Anh Tuấn – 2541864024	Thiết kế CSDL, migration, index, hàm PL/pgSQL; backend Node.js; benchmark; viết chương 2, 3, 5	100%
2	Võ Bùi Duy Thông – 2541864019	Phân tích nghiệp vụ; giao diện Next.js (khách hàng + quản trị); Docker; viết chương 1, 4, kết luận; slide	100%

MỤC LỤC

MỞ ĐẦU 1
1. Lý do chọn đề tài 1
2. Mục tiêu của đề tài 1
3. Đối tượng và phạm vi nghiên cứu 2
4. Phương pháp thực hiện 2
5. Bố cục tiểu luận 3
CHƯƠNG 1: TỔNG QUAN VỀ POSTGRESQL 4
1.1. Lịch sử và giấy phép 4
1.2. Vị trí trên thị trường 4
1.3. Hệ sinh thái tính năng và extension 5
1.4. Lý do lựa chọn PostgreSQL cho hệ thống bán hàng trang sức 6
CHƯƠNG 2: PHÂN TÍCH NGHIỆP VỤ VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU 7
2.1. Mô tả nghiệp vụ 7
2.2. Sơ đồ thực thể – quan hệ (ERD) 8
2.3. Mô tả chi tiết các bảng 9
2.3.1. Bảng products – sản phẩm 9
2.3.2. Bảng orders – đơn hàng (phân vùng theo tháng) 10
2.3.3. Bảng order_items – chi tiết đơn hàng 11
2.3.4. Các bảng còn lại 11
2.4. Ràng buộc toàn vẹn và trigger 12
2.5. Tổ chức migration 13
CHƯƠNG 3: CÁC TÍNH NĂNG NỔI BẬT CỦA POSTGRESQL ÁP DỤNG TRONG HỆ THỐNG 14
3.1. JSONB – thuộc tính sản phẩm linh hoạt 14
3.1.1. Bài toán 14
3.1.2. Giải pháp PostgreSQL 14
3.1.3. Kết quả 15
3.2. Full-text search tiếng Việt với unaccent, tsvector và pg_trgm 15
3.2.1. Bài toán 15
3.2.2. Giải pháp PostgreSQL 15

3.2.3. Kết quả....16
3.3. pgvector – gợi ý sản phẩm tương tự....17
3.3.1. Bài toán....17
3.3.2. Giải pháp PostgreSQL....17
3.3.3. Kết quả....18
3.4. Table partitioning và BRIN index cho bảng đơn hàng....20
3.4.1. Bài toán....20
3.4.2. Giải pháp PostgreSQL....20
3.4.3. Kết quả....21
3.5. Transaction và SELECT … FOR UPDATE – chống bán quá tồn kho....22
3.5.1. Bài toán....22
3.5.2. Giải pháp PostgreSQL....22
3.5.3. Kết quả....24
3.6. Materialized view và LISTEN/NOTIFY cho dashboard quản trị....24
3.6.1. Bài toán....24
3.6.2. Giải pháp PostgreSQL....24
3.6.3. Kết quả....26
3.7. Hệ thống chỉ mục và các loại index đặc biệt....26
CHƯƠNG 4: XÂY DỰNG ỨNG DỤNG....28
4.1. Kiến trúc tổng thể....28
4.2. Backend: SQL thuần với Node.js....29
4.2.1. Lớp truy cập dữ liệu....29
4.2.2. Trình chạy migration....29
4.2.3. Các nhóm API....30
4.3. Frontend Next.js....30
4.4. Giao diện khách hàng....31
4.5. Giao diện quản trị....33
4.6. Triển khai bằng Docker Compose....34
CHƯƠNG 5: KIỂM THỬ, BENCHMARK VÀ GIÁM SÁT....36
5.1. Dữ liệu thử nghiệm: 1 triệu đơn hàng....36
5.2. Công cụ đo: bench.js....36
5.3. Kết quả: có và không có index trên 1 triệu đơn hàng....37
5.4. Kiểm chứng chống bán quá tồn kho....39

5.5. Thông lượng với pgbench....40
5.6. Giám sát....40
KẾT LUẬN....42
1. Kết quả đạt được....42
2. Hạn chế....43
3. Hướng phát triển....43
TÀI LIỆU THAM KHẢO....44
PHỤ LỤC....45
Phụ lục A. Cài đặt và chạy trên máy cá nhân....45
Phụ lục B. Kết nối DBeaver và các câu lệnh demo....45
Phụ lục C. Cấu trúc mã nguồn....46

DANH MỤC HÌNH ẢNH

Hình 2.1: Sơ đồ thực thể – quan hệ của hệ thống.....8
Hình 3.1: Gõ sai chính tả – hệ thống tự chuyển sang tìm gần đúng bằng trigram.....17
Hình 3.2: Trang chi tiết sản phẩm với khối "Có thể bạn cũng thích" từ pgvector.....19
Hình 3.3: Partition pruning: chỉ quét 1 partition (trái) so với Append toàn bộ partition (phải).....21
Hình 3.4: Luồng xử lý trong hàm place_order().....23
Hình 3.5: Thông báo "Sắp hết hàng" đẩy từ PostgreSQL (pg_notify) sang trình duyệt quản trị.....26
Hình 4.1: Kiến trúc triển khai của hệ thống.....28
Hình 4.2: Danh sách sản phẩm lọc theo danh mục Nhẫn và chất liệu Vàng 18K (JSONB).....32
Hình 4.3: Giỏ hàng và biểu mẫu đặt hàng.....32
Hình 4.4: Dashboard quản trị (dữ liệu từ materialized view).....33
Hình 4.5: Màn hình sửa sản phẩm – thêm/xoá thuộc tính JSONB và nhập kho.....34
Hình 5.1: Thống kê câu SQL từ pg_stat_statements trên trang quản trị.....39

DANH MỤC BẢNG BIỂU

Bảng 1.1: Top 5 hệ quản trị CSDL theo DB-Engines (2026) [1].....4
Bảng 1.2: So sánh PostgreSQL với MySQL và MongoDB ở các nhu cầu của đề tài.....5
Bảng 2.1: Danh sách chức năng và yêu cầu dữ liệu.....7
Bảng 2.2: Cấu trúc bảng products.....9
Bảng 2.3: Cấu trúc bảng orders.....10
Bảng 2.4: Cấu trúc bảng order_items.....11
Bảng 2.5: Các bảng phụ trợ.....11
Bảng 2.6: Các file migration.....13
Bảng 3.1: Các index và cột sinh tự động khác trong hệ thống.....26
Bảng 4.1: Công nghệ sử dụng.....28
Bảng 5.1: Môi trường và dữ liệu thử nghiệm.....36
Bảng 5.2: Thời gian thực thi có và không có index (trung vị 5 lần, 1.035.853 đơn hàng).....37
Bảng 5.3: Kích thước các index trên bảng orders và products.....38
Bảng 5.4: Kết quả pgbench (máy ảo 2 vCPU, cấu hình PostgreSQL mặc định).....40

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Ý nghĩa
CSDL	Database	Cơ sở dữ liệu
HQTCSDL	DBMS – Database Management System	Hệ quản trị cơ sở dữ liệu
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
ORM	Object-Relational Mapping	Ánh xạ đối tượng – quan hệ (thư viện trung gian giữa code và SQL)
API	Application Programming Interface	Giao diện lập trình ứng dụng
JSON / JSONB	JavaScript Object Notation (Binary)	Định dạng dữ liệu dạng cặp khoá–giá trị; JSONB là dạng nhị phân có index trong PostgreSQL
FTS	Full-Text Search	Tìm kiếm toàn văn
GIN	Generalized Inverted Index	Chỉ mục đảo ngược tổng quát
BRIN	Block Range Index	Chỉ mục theo khoảng khối
HNSW	Hierarchical Navigable Small World	Thuật toán index tìm láng giềng gần nhất cho vector
MVCC	Multi-Version Concurrency Control	Điều khiển đồng thời đa phiên bản
ACID	Atomicity, Consistency, Isolation, Durability	Bốn tính chất của giao dịch
SSE	Server-Sent Events	Cơ chế máy chủ đẩy sự kiện xuống trình duyệt
JWT	JSON Web Token	Chuỗi xác thực phiên đăng nhập

MỞ ĐẦU

1. Lý do chọn đề tài

Trong hoạt động kinh doanh trang sức, dữ liệu có nhiều đặc thù: mỗi loại sản phẩm (nhẫn, bông tai, dây chuyền, vòng tay…) có bộ thuộc tính khác nhau (ni nhẫn, carat, chiều dài dây, loại đá…); tồn kho mỗi mã thường rất ít, có khi chỉ một chiếc; khách hàng tìm kiếm bằng tiếng Việt có dấu hoặc không dấu và hay gõ sai chính tả; đơn hàng tích luỹ ngày càng lớn theo thời gian; người quản lý cần báo cáo doanh thu nhanh và được cảnh báo kịp thời khi hàng sắp hết. Một hệ quản trị CSDL quan hệ thông thường chỉ với bảng – khoá – index B-tree giải quyết được bài toán nhưng không tối ưu, thường phải kéo thêm nhiều thành phần bên ngoài (Elasticsearch để tìm kiếm, Redis để cache, hàng đợi để thông báo…).

PostgreSQL là HQTCSDL mã nguồn mở đang đứng thứ 4 bảng xếp hạng DB-Engines và là hệ CSDL có mức tăng điểm cao nhất nửa đầu năm 2026 [1][2]. Điểm mạnh của PostgreSQL không chỉ ở việc tuân thủ chuẩn SQL mà còn ở hệ sinh thái tính năng và extension phong phú: JSONB, full-text search, pgvector, partitioning, nhiều loại index chuyên biệt, materialized view, LISTEN/NOTIFY… Những tính năng này cho phép giải quyết trọn vẹn các đặc thù nêu trên ngay bên trong một CSDL duy nhất.

Vì vậy nhóm chọn đề tài **"Xây dựng hệ thống bán hàng trang sức với PostgreSQL"** để vừa nghiên cứu các tính năng nổi bật của PostgreSQL, vừa vận dụng vào một ứng dụng hoàn chỉnh có giao diện cho khách hàng và người quản trị.

2. Mục tiêu của đề tài

- Tìm hiểu tổng quan PostgreSQL: lịch sử, kiến trúc, vị trí trên thị trường và các tính năng khác biệt so với các HQTCSDL phổ biến.
- Phân tích nghiệp vụ bán hàng trang sức và thiết kế CSDL quan hệ phù hợp trên PostgreSQL, có đầy đủ ràng buộc và chỉ mục.

- Vận dụng và đánh giá **6 tính năng nổi bật** của PostgreSQL gắn với 6 bài toán nghiệp vụ cụ thể: JSONB; full-text search tiếng Việt; pgvector; partitioning + BRIN; transaction với khoá dòng; materialized view + LISTEN/NOTIFY.
- Xây dựng ứng dụng web hoàn chỉnh (trang bán hàng + trang quản trị) với backend Node.js dùng SQL thuần (không ORM) và frontend Next.js; đóng gói triển khai bằng Docker.
- Đo lường hiệu năng bằng công cụ đo tự viết (EXPLAIN ANALYZE), pgbench và giám sát bằng pg_stat_statements ngay trong trang quản trị.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng: HQTCSDL PostgreSQL phiên bản 16 và các extension đi kèm (pg_trgm, unaccent, pgvector, pg_stat_statements).

Phạm vi nghiệp vụ: bán trang sức trực tuyến ở mức cơ bản: quản lý danh mục – sản phẩm – tồn kho, tìm kiếm, giỏ hàng, đặt hàng, duyệt/huỷ đơn, báo cáo doanh thu. Không đi sâu vào thanh toán trực tuyến, vận chuyển, chương trình khuyến mãi hay nhiều kho hàng. Trọng tâm của tiểu luận là phần cơ sở dữ liệu; ứng dụng được xây dựng đủ để minh hoạ và kiểm chứng.

4. Phương pháp thực hiện

- Nghiên cứu tài liệu chính thức của PostgreSQL và các extension; tham khảo bảng xếp hạng DB-Engines.
- Phân tích nghiệp vụ, thiết kế ERD, viết lược đồ bằng SQL thuần theo từng file migration.
- Xây dựng ứng dụng theo mô hình 3 lớp (trình duyệt – API – CSDL), đưa logic nghiệp vụ quan trọng (đặt hàng) vào hàm PL/pgSQL trong CSDL.
- Sinh dữ liệu đủ lớn (hơn 3.000 sản phẩm, 1 triệu đơn hàng, 37 partition) và viết công cụ đo riêng so sánh có/không index bằng EXPLAIN ANALYZE; kiểm chứng tính đúng đắn bằng kịch bản đặt hàng đồng thời với pgbench.

5. Bố cục tiểu luận

Ngoài phần mở đầu, kết luận, tài liệu tham khảo và phụ lục, tiểu luận gồm 5 chương:

- **Chương 1 – Tổng quan về PostgreSQL:** lịch sử, kiến trúc, vị trí trên thị trường, lý do lựa chọn.
- **Chương 2 – Phân tích nghiệp vụ và thiết kế cơ sở dữ liệu:** chức năng, ERD, mô tả từng bảng, ràng buộc, migration.
- **Chương 3 – Các tính năng nổi bật của PostgreSQL áp dụng trong hệ thống:** 6 tính năng chính và hệ thống chỉ mục.
- **Chương 4 – Xây dựng ứng dụng:** kiến trúc, backend, frontend, giao diện, triển khai Docker.
- **Chương 5 – Kiểm thử, benchmark và giám sát:** dữ liệu 1 triệu đơn, công cụ đo, kết quả có/không index, kiểm chứng đồng thời, giám sát.

CHƯƠNG 1: TỔNG QUAN VỀ POSTGRESQL

1.1. Lịch sử và giấy phép

PostgreSQL bắt nguồn từ dự án POSTGRES của giáo sư Michael Stonebraker tại Đại học California, Berkeley (1986), kế thừa hệ Ingres. Năm 1996 dự án đổi tên thành PostgreSQL để nhấn mạnh việc hỗ trợ SQL và được phát triển bởi cộng đồng PostgreSQL Global Development Group cho đến nay. Phiên bản 16 phát hành tháng 9/2023, phiên bản 17 tháng 9/2024; mỗi năm có một phiên bản lớn và các bản vá định kỳ [3].

PostgreSQL phát hành theo giấy phép PostgreSQL License (tương tự BSD/MIT): được dùng, sửa, phân phối miễn phí kể cả trong sản phẩm thương mại, không có phiên bản "Enterprise" thu phí. Đây là lý do nhiều nền tảng cloud (Amazon RDS/Aurora, Google Cloud SQL/AlloyDB, Azure Database, Supabase, Neon…) chọn PostgreSQL làm lõi.

1.2. Vị trí trên thị trường

Theo bảng xếp hạng DB-Engines (tổng hợp từ mức độ thảo luận, tìm kiếm, tuyển dụng, tài liệu…), PostgreSQL ổn định ở vị trí thứ 4 sau Oracle, MySQL và Microsoft SQL Server, nhưng là hệ CSDL duy nhất trong top 4 liên tục **tăng điểm** trong khi ba hệ còn lại giảm. PostgreSQL từng được DB-Engines bình chọn là "DBMS of the Year" các năm 2017, 2018, 2020 và 2023; trong nửa đầu năm 2026 PostgreSQL tiếp tục dẫn đầu về mức tăng điểm (+21,97 điểm) [1][2]. Khảo sát Stack Overflow Developer Survey các năm gần đây cũng xếp PostgreSQL là CSDL được lập trình viên sử dụng và yêu thích nhiều nhất [4].

Bảng 1.1: Top 5 hệ quản trị CSDL theo DB-Engines (2026) [1]

Hạng	Hệ quản trị CSDL	Mô hình	Giấy phép	Xu hướng điểm
1	Oracle	Quan hệ, đa mô hình	Thương mại	Giảm nhẹ

Hạng	Hệ quản trị CSDL	Mô hình	Giấy phép	Xu hướng điểm
2	MySQL	Quan hệ	GPL / thương mại	Giảm
3	Microsoft SQL Server	Quan hệ	Thương mại	Giảm
4	PostgreSQL	Quan hệ, đa mô hình	Mã nguồn mở (PostgreSQL License)	Tăng
5	MongoDB	Tài liệu (NoSQL)	SSPL	Tăng nhẹ

1.3. Hệ sinh thái tính năng và extension

PostgreSQL có giao dịch ACID đầy đủ với MVCC (người đọc không chặn người ghi), nhật ký WAL và DDL nằm trong transaction. Khác với các HQTCSDL "chỉ quan hệ", PostgreSQL được gọi là hệ CSDL **đa mô hình**: ngoài bảng và SQL chuẩn, nó hỗ trợ sẵn kiểu dữ liệu JSONB, mảng, kiểu khoảng (range), kiểu hình học, tìm kiếm toàn văn, nhiều phương pháp index (B-tree, Hash, GIN, GiST, SP-GiST, BRIN), bảng phân vùng khai báo, materialized view, trigger, hàm lưu trữ bằng nhiều ngôn ngữ (PL/pgSQL, Python, JavaScript…) và cơ chế thông báo LISTEN/NOTIFY. Cơ chế **extension** cho phép bổ sung kiểu dữ liệu, toán tử và loại index mới mà không sửa lõi; những extension tiêu biểu là PostGIS (không gian địa lý), pgvector (vector cho AI), pg_trgm (tìm kiếm gần đúng), TimescaleDB (chuỗi thời gian), Citus (phân tán).

Bảng 1.2: So sánh PostgreSQL với MySQL và MongoDB ở các nhu cầu của đề tài

Nhu cầu	PostgreSQL	MySQL	MongoDB
Thuộc tính linh hoạt	JSONB + GIN index, toán tử @>, ->>	JSON (index phải tạo cột ảo)	Tài liệu BSON tự nhiên
Tìm kiếm toàn	tsvector/tsquery, xếp	FULLTEXT index	Text index cơ

Nhu cầu	PostgreSQL	MySQL	MongoDB
văn	hạng, nhiều ngôn ngữ, kết hợp pg_trgm	(hạn chế tiếng Việt)	bản / Atlas Search
Vector cho AI	pgvector (HNSW, IVFFlat)	Chỉ từ MySQL 9 (HeatWave)	Atlas Vector Search (cloud)
Phân vùng bảng	Khai báo RANGE/LIST/HASH, partition pruning	Có, nhưng hạn chế khoá ngoại	Sharding
Giao dịch	ACID đầy đủ, DDL trong transaction	ACID (InnoDB), DDL không rollback	Giao dịch đa tài liệu từ 4.0
Thông báo sự kiện	LISTEN/NOTIFY tích hợp	Không có	Change Streams
Giấy phép	Mở hoàn toàn	GPL/thương mại (Oracle)	SSPL

1.4. Lý do lựa chọn PostgreSQL cho hệ thống bán hàng trang sức

Từ phân tích trên, PostgreSQL phù hợp với đề tài vì: (1) vẫn là CSDL quan hệ nên bảo đảm tính toàn vẹn cho đơn hàng – tồn kho – thanh toán; (2) JSONB giải quyết thuộc tính khác nhau giữa các loại trang sức; (3) full-text search và pg_trgm xử lý tìm kiếm tiếng Việt ngay trong CSDL; (4) pgvector cho phép thêm tính năng gợi ý sản phẩm theo hướng AI mà không cần hệ thống riêng; (5) partitioning, BRIN, materialized view đáp ứng dữ liệu đơn hàng lớn dần; (6) LISTEN/NOTIFY thay thế hàng đợi thông báo. Tất cả nằm trong một hệ thống duy nhất, miễn phí, dễ vận hành bằng Docker.

CHƯƠNG 2: PHÂN TÍCH NGHIỆP VỤ VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU

2.1. Mô tả nghiệp vụ

Hệ thống phục vụ hai nhóm người dùng:

- **Khách hàng** (không cần đăng nhập): xem danh mục, lọc sản phẩm theo chất liệu/loại đá/giá, tìm kiếm, xem chi tiết và sản phẩm tương tự, thêm vào giỏ, đặt hàng bằng họ tên – số điện thoại – địa chỉ, tra cứu đơn hàng.
- **Quản trị viên** (đăng nhập): xem dashboard doanh thu, top sản phẩm, cảnh báo sắp hết hàng theo thời gian thực; quản lý sản phẩm và thuộc tính; nhập kho; duyệt, chuyển trạng thái, huỷ đơn; theo dõi hiệu năng CSDL.

Quy trình đơn hàng: `pending` (khách vừa đặt) → `confirmed` (đã gọi xác nhận) → `shipping` → `completed`; có thể `cancelled` ở hai bước đầu, khi đó tồn kho được trả lại. Đơn từ 5 triệu đồng miễn phí vận chuyển, dưới 5 triệu phí 30.000 đ.

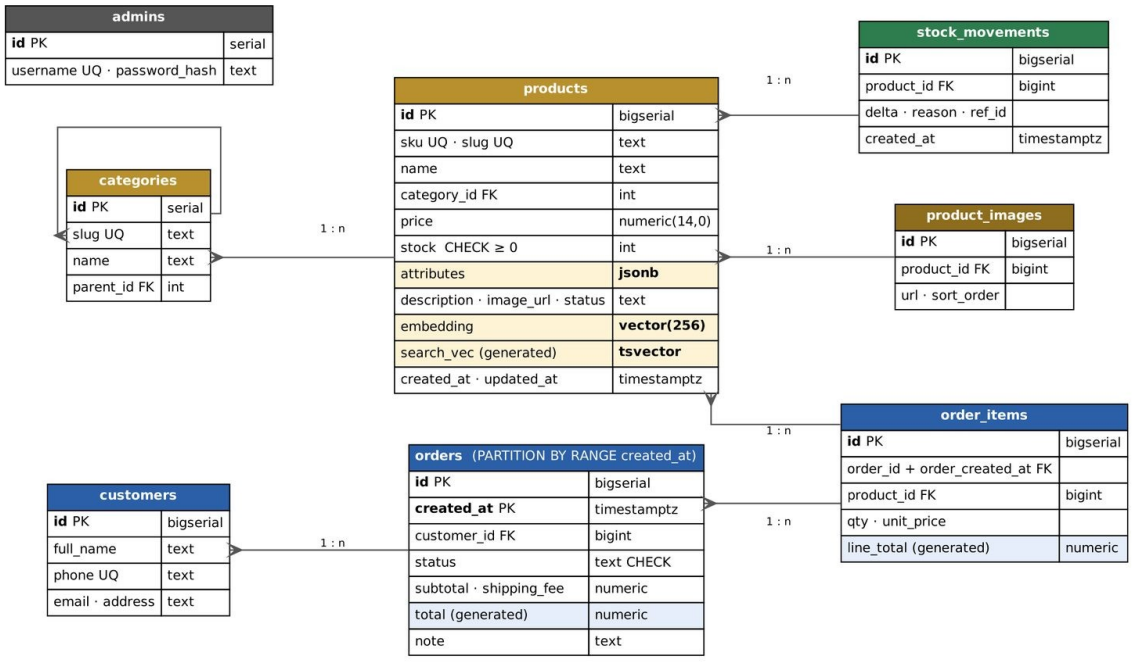
Bảng 2.1: Danh sách chức năng và yêu cầu dữ liệu

STT	Chức năng	Tác nhân	Yêu cầu dữ liệu / ràng buộc
1	Xem & lọc sản phẩm	Khách	Lọc theo danh mục, chất liệu, đá, giới tính, khoảng giá; phân trang; sắp xếp
2	Tìm kiếm	Khách	Không dấu / có dấu đều khớp; chịu lỗi chính tả; gợi ý khi gõ
3	Sản phẩm tương tự	Khách	Gợi ý 8 sản phẩm gần nhất về chất liệu, loại, đá
4	Đặt hàng	Khách	Không bán quá tồn kho khi nhiều người đặt cùng lúc; lưu giá tại thời điểm mua
5	Tra cứu đơn	Khách	Theo mã đơn + số điện thoại
6	Dashboard	Admin	Doanh thu theo ngày/tháng/danh mục, top sản phẩm; phải nhanh dù đơn hàng lớn

STT	Chức năng	Tác nhân	Yêu cầu dữ liệu / ràng buộc
7	Cảnh báo hết hàng	Admin	Thông báo ngay khi tồn < 3 mà không cần tải lại trang
8	Quản lý sản phẩm, kho	Admin	Thuộc tính thêm tuỳ ý; nhật ký xuất nhập kho
9	Xử lý đơn	Admin	Chuyển trạng thái; huỷ phải trả kho trong cùng giao dịch
10	Giám sát CSDL	Admin	Thống kê câu SQL, index, partition; chạy EXPLAIN so sánh

2.2. Sơ đồ thực thể – quan hệ (ERD)

Lược đồ gồm 8 bảng, chia thành ba nhóm: danh mục – sản phẩm (categories, products, product_images), bán hàng (customers, orders, order_items), vận hành (stock_movements, admins). Các cột tô màu là nơi áp dụng tính năng đặc biệt của PostgreSQL.



Hình 2.1: Sơ đồ thực thể – quan hệ của hệ thống

2.3. Mô tả chi tiết các bảng

2.3.1. Bảng products – sản phẩm

Đây là bảng trung tâm, chứa ba cột đặc biệt: `attributes` (JSONB), `search_vec` (tsvector sinh tự động) và `embedding` (vector 256 chiều).

Bảng 2.2: Cấu trúc bảng products

Cột	Kiểu	Ràng buộc	Mô tả
id	BIGSERIAL	PK	Mã sản phẩm nội bộ
sku	TEXT	UNIQUE	Mã hàng hiển thị (`NK-001`)
slug	TEXT	UNIQUE	Định danh trên URL
name	TEXT	NOT NULL	Tên sản phẩm
category_id	INT	FK → categories	Danh mục
price	NUMERIC(14,0)	CHECK ≥ 0	Giá bán (VNĐ, không số lẻ)
stock	INT	CHECK ≥ 0	Tồn kho; ràng buộc cuối cùng chống bán âm
attributes	JSONB	DEFAULT {}	Thuộc tính linh hoạt: metal, stone, carat, ring_size, length_cm…
description	TEXT		Mô tả
image_url	TEXT		Ảnh đại diện
status	TEXT	CHECK IN (active, hidden, discontinued)	Trạng thái bán
embedding	VECTOR(256)		Vector đặc trưng cho gợi ý sản phẩm tương tự (pgvector)
search_vec	TSVECTOR	GENERATED	Tự tính từ name (trọng số A) + description (trọng số B), đã bỏ dấu

Cột	Kiểu	Ràng buộc	Mô tả
		STORED	
created_at, updated_at	TIMESTAMPTZ	DEFAULT now()	updated_at cập nhật bằng trigger

2.3.2. Bảng orders – đơn hàng (phân vùng theo tháng)

Bảng `orders` được khai báo `PARTITION BY RANGE (created_at)`, mỗi tháng một partition con (`orders_2026_01` … `orders_2026_12`) và một partition mặc định. Khoá chính bắt buộc chứa cột phân vùng nên là `(id, created_at)`.

Bảng 2.3: Cấu trúc bảng orders

Cột	Kiểu	Ràng buộc	Mô tả
id	BIGSERIAL	PK (cùng created_at)	Mã đơn
created_at	TIMESTAMPTZ	PK, khoá phân vùng	Thời điểm đặt
customer_id	BIGINT	FK → customers	
status	TEXT	CHECK 5 giá trị	pending / confirmed / shipping / completed / cancelled
subtotal	NUMERIC(14,0)		Tiền hàng
shipping_fee	NUMERIC(14,0)		Phí vận chuyển
total	NUMERIC(14,0)	GENERATED ALWAYS AS (subtotal +	Tổng tiền do PostgreSQL tính, ứng dụng không thể ghi sai

Cột	Kiểu	Ràng buộc	Mô tả
		shipping_ fee)	
note	TEXT		Ghi chú của khách

2.3.3. Bảng order_items – chi tiết đơn hàng

Bảng 2.4: Cấu trúc bảng order_items

Cột	Kiểu	Ràng buộc	Mô tả
id	BIGSERIAL	PK	
order_id, order_created_ at	BIGINT, TIMESTAM PTZ	FK → orders(id, created_at)	Khoá ngoại tới bảng phân vùng phải trỏ đủ hai cột
product_id	BIGINT	FK → products	
qty	INT	CHECK > 0	Số lượng
unit_price	NUMERIC(1 4,0)		Giá tại thời điểm mua (giá sản phẩm có thể đổi sau)
line_total	NUMERIC(1 4,0)	GENERA TED (qty × unit_price)	Thành tiền

2.3.4. Các bảng còn lại

Bảng 2.5: Các bảng phụ trợ

Bảng	Vai trò và các cột chính
categories	Danh mục: id, slug (UNIQUE, dùng trên URL), name,

Bảng	Vai trò và các cột chính
	parent_id (FK tự tham chiếu → danh mục con, ví dụ Nhẫn → Nhẫn cưới).
customers	Khách hàng: id, full_name, phone (UNIQUE – khoá nhận diện, dùng upsert khi đặt hàng), email (expression index lower(email)), address.
product_images	Ảnh phụ của sản phẩm: product_id (FK, ON DELETE CASCADE), url, sort_order.
stock_movements	Nhật ký xuất/nhập kho để truy vết vì sao tồn kho thay đổi: product_id, delta (âm = bán, dương = nhập/huỷ đơn), reason (order/cancel/import/adjust), ref_id (mã đơn liên quan), created_at.
admins	Tài khoản quản trị: username (UNIQUE), password_hash (bcrypt), full_name.
schema_migrations	Bảng kỹ thuật ghi nhớ file migration nào đã chạy (filename, applied_at).

2.4. Ràng buộc toàn vẹn và trigger

- **Khoá ngoại** đầy đủ giữa các bảng; `product_images` xoá theo sản phẩm (CASCADE), `order_items` xoá theo đơn.
- **CHECK:** `price ≥ 0`, `stock ≥ 0`, `qty > 0`, `status` thuộc tập giá trị cho phép.
- **UNIQUE:** slug, sku, phone, username.
- **Generated column:** `orders.total`, `order_items.line_total`, `products.search_vec` – dữ liệu dẫn xuất luôn đúng, không phụ thuộc ứng dụng.
- **Trigger:** `set_updated_at` (cập nhật updated_at), `notify_low_stock` (bắn NOTIFY khi stock < 3), `notify_new_order` (bắn NOTIFY khi có đơn mới).

– **Hàm nghiệp vụ:** `place_order()` và `cancel_order()` gói trọn logic đặt/huỷ đơn trong một giao dịch (trình bày ở chương 3).

2.5. Tổ chức migration

Lược đồ được viết bằng SQL thuần trong 6 file đánh số thứ tự; mỗi file tập trung vào một chủ đề để dễ trình bày và dễ bảo trì. Trình chạy migration (viết khoảng 60 dòng Node.js, mục 4.3) ghi nhớ file đã chạy trong bảng `schema_migrations` và chạy từng file trong một giao dịch – nhờ PostgreSQL hỗ trợ DDL trong transaction nên nếu một file lỗi giữa chừng thì toàn bộ thay đổi của file đó được huỷ.

Bảng 2.6: Các file migration

File	Nội dung
001_extensions.sql	Bật pg_trgm, unaccent, vector, pg_stat_statements; hàm `vi_unaccent()` IMMUTABLE
002_core_tables.sql	categories, products (JSONB, tsvector, vector), product_images, customers, admins, trigger updated_at
003_orders_partitioned.sql	orders phân vùng theo tháng, hàm tạo partition, order_items, stock_movements
004_indexes.sql	Toàn bộ index: B-tree, GIN (JSONB, tsvector, trigram), HNSW, BRIN, covering, partial, expression
005_order_transaction.sql	Hàm place_order(), cancel_order(); trigger NOTIFY low_stock, new_order
006_reports_matview.sql	Materialized view báo cáo, hàm refresh, các view tiện ích

CHƯƠNG 3: CÁC TÍNH NĂNG NỔI BẬT CỦA POSTGRESQL ÁP DỤNG TRONG HỆ THỐNG

Chương này trình bày 6 tính năng được chọn, mỗi tính năng theo cùng một cấu trúc: bài toán nghiệp vụ → cách PostgreSQL giải quyết → mã SQL thực tế trong hệ thống → kết quả. Các số liệu đo được tổng hợp ở chương 5.

3.1. JSONB – thuộc tính sản phẩm linh hoạt

3.1.1. Bài toán

Nhẫn có ni (size), carat, nước màu, độ sạch; dây chuyền có chiều dài; bông tai có kiểu khoá; vòng cẩm thạch có đường kính và giấy kiểm định. Nếu dùng mô hình quan hệ thuần tuý có hai cách: (a) thêm hàng chục cột vào bảng products, phần lớn NULL; (b) tạo bảng EAV (entity–attribute–value) khiến truy vấn phải JOIN nhiều lần và mất kiểu dữ liệu. Cả hai đều khó mở rộng khi cửa hàng nhập loại sản phẩm mới.

3.1.2. Giải pháp PostgreSQL

Kiểu **JSONB** lưu tài liệu JSON dưới dạng nhị phân đã phân tích: truy xuất khoá không cần parse lại, hỗ trợ toán tử chứa `@>`, tồn tại khoá `?`, lấy giá trị `->>`, và quan trọng nhất là **đánh index GIN** được. Lớp `jsonb_path_ops` tạo index nhỏ và nhanh hơn cho toán tử `@>` – đúng nhu cầu lọc "chất liệu = Vàng 18K và đá = Kim cương".

```
-- 002_core_tables.sql
attributes JSONB NOT NULL DEFAULT '{}'::jsonb,

-- 004_indexes.sql
CREATE INDEX idx_products_attributes ON products USING GIN (attributes
jsonb_path_ops);

-- Dữ liệu mẫu: mỗi loại trang sức một bộ thuộc tính
{"metal":"Vàng trắng 18K","weight_g":2.8,"ring_size":12,"stone":"Kim
cương","carat":0.5,"color":"F","clarity":"VS1"}
{"metal":"Vàng
24K","weight_g":37.5,"length_cm":50,"style":"Xích","gender":"Nam"}
{"stone":"Cẩm
thạch","color":"Xanh","diameter_mm":56,"origin":"Myanmar","certified":true}
```

Ở tầng API, các tham số lọc trên URL (chất liệu, đá, giới tính) được gom thành một đối tượng JSON rồi so khớp bằng `@>` – một câu SQL duy nhất cho mọi tổ hợp

thuộc tính; bộ lọc bên trái trang danh sách cũng được sinh từ JSONB bằng `attributes->>'metal'` và `GROUP BY`.

3.1.3. Kết quả

EXPLAIN ANALYZE cho thấy planner dùng `Bitmap Index Scan on idx_products_attributes`, truy vấn lọc 2 thuộc tính trên 3.033 sản phẩm mất **0,06 ms** so với **2,1 ms** khi quét tuần tự (nhanh hơn ~30 lần). Trang quản trị cho phép thêm/xoá thuộc tính tuỳ ý mà không cần ALTER TABLE (Hình 4.5).

3.2. Full-text search tiếng Việt với unaccent, tsvector và pg_trgm

3.2.1. Bài toán

Khách gõ "nhan kim cuong 18k" (không dấu) phải tìm được "Nhẫn kim cương vàng trắng 18K Halo"; gõ sai "nhan kim cuog solitare" vẫn nên ra "Nhẫn kim cương vàng 18K Solitaire". `LIKE '%...%'` không dùng được index B-tree, không xử lý dấu và không xếp hạng kết quả.

3.2.2. Giải pháp PostgreSQL

PostgreSQL có bộ tìm kiếm toàn văn tích hợp: văn bản được chuyển thành **tsvector** (danh sách từ vựng đã chuẩn hoá kèm vị trí và trọng số), câu hỏi thành **tsquery**, so khớp bằng toán tử `@@`, xếp hạng bằng `ts_rank`, tô đậm bằng `ts_headline`. Vì PostgreSQL chưa có từ điển tiếng Việt, nhóm dùng cấu hình `simple` kết hợp extension **unaccent** để bỏ dấu cả dữ liệu lẫn câu hỏi. Hàm `unaccent()` gốc chỉ là STABLE nên phải bọc thành hàm IMMUTABLE để được dùng trong index và generated column:

```
-- 001_extensions.sql
CREATE OR REPLACE FUNCTION vi_unaccent(text) RETURNS text
LANGUAGE sql IMMUTABLE PARALLEL SAFE STRICT
AS $$ SELECT lower(public.unaccent('public.unaccent', $1)) $$;

-- 002_core_tables.sql: tsvector là GENERATED COLUMN, tự cập nhật khi
INSERT/UPDATE
search_vec TSVECTOR GENERATED ALWAYS AS (
  setweight(to_tsvector('simple', vi_unaccent(name)), 'A') ||
  setweight(to_tsvector('simple', vi_unaccent(coalesce(description, ''))),
'B')
) STORED,
```

```
-- 004_indexes.sql
CREATE INDEX idx_products_search_vec ON products USING GIN (search_vec);
CREATE INDEX idx_products_name_trgm  ON products USING GIN (vi_unaccent(name)
gin_trgm_ops);
```

Để chịu được lỗi chính tả, nhóm dùng thêm extension **pg_trgm**: chia chuỗi thành các bộ ba ký tự (trigram) và đo độ tương đồng. API tìm kiếm chạy theo hai tầng: tầng 1 full-text (chính xác, có xếp hạng); nếu không có kết quả thì tầng 2 trigram với toán tử `%` (similarity vượt ngưỡng). Cả hai tầng đều chạy trên index GIN.

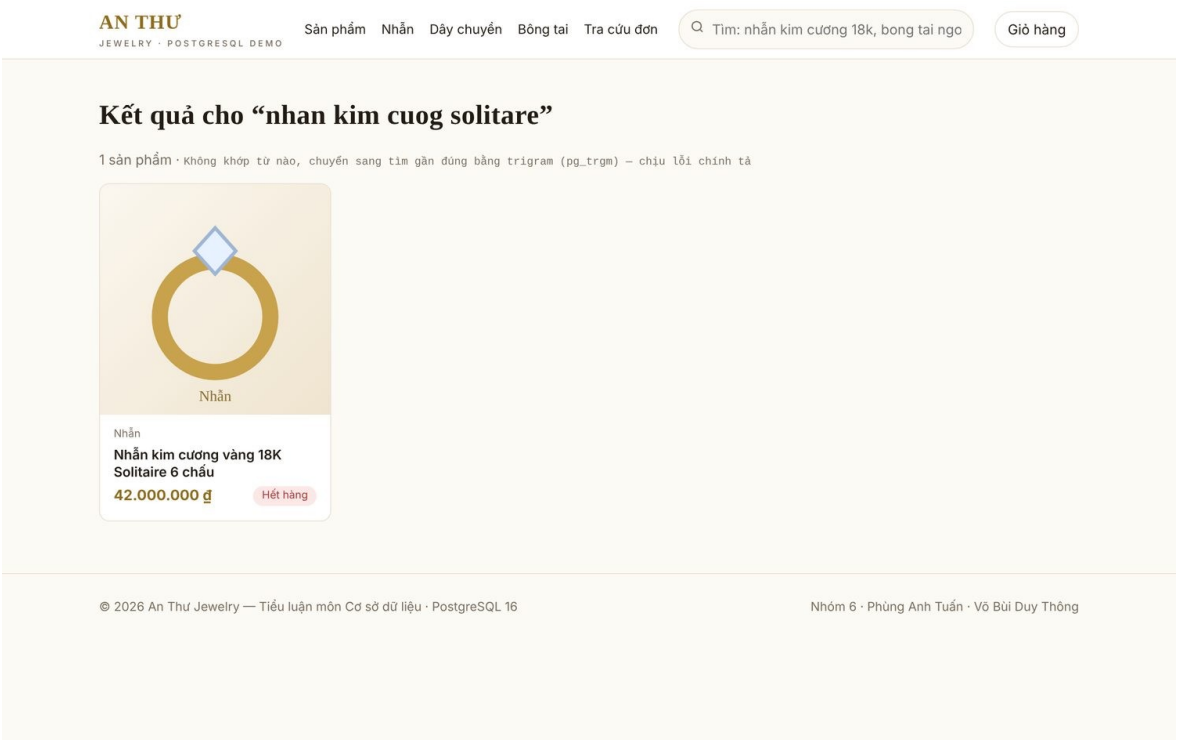
```
-- Tầng 1: full-text search (api/src/routes/search.js)
SELECT p.name, ts_rank(p.search_vec, q) AS rank,
       ts_headline('simple', p.name, q, 'StartSel=<b>, StopSel=</b>') AS
headline
FROM products p, plainto_tsquery('simple', vi_unaccent($1)) q
WHERE p.status = 'active' AND p.search_vec @@ q
ORDER BY rank DESC LIMIT 24;

-- Tầng 2: tìm gần đúng bằng trigram
SELECT p.name, similarity(vi_unaccent(p.name), vi_unaccent($1)) AS rank
FROM products p
WHERE vi_unaccent(p.name) % vi_unaccent($1)
ORDER BY rank DESC LIMIT 24;

-- Gợi ý khi gõ: khớp tiền tố 'nhan:* & kim:*'
WHERE search_vec @@ to_tsquery('simple', vi_unaccent('nhan:* & kim:*'))
```

3.2.3. Kết quả

Tìm "nhan kim cuong 18k" trả về 3 nhẫn kim cương viết tay xếp đầu rồi đến các mẫu sinh tự động; gõ sai "nhan kim cuog solitare" tầng full-text không khớp, tầng trigram tìm đúng "Nhẫn kim cương vàng 18K Solitaire 6 chấu" với similarity 0,44 (Hình 3.1). Thanh tìm kiếm gợi ý sau 200 ms mỗi lần gõ nhờ prefix search. Full-text có index nhanh hơn quét tuần tự ~9 lần, trigram ~2,6 lần.



Hình 3.1: Gõ sai chính tả – hệ thống tự chuyển sang tìm gần đúng bằng trigram

3.3. pgvector – gợi ý sản phẩm tương tự

3.3.1. Bài toán

Khi khách xem “Nhẫn kim cương vàng trắng 18K Halo”, cửa hàng muốn gợi ý các sản phẩm “giống”: cùng loại nhẫn, cùng vàng trắng, cùng đính kim cương, giá gần nhau. Quy tắc “cùng danh mục” quá thô; viết tay các điều kiện lọc thì cứng nhắc. Cách tiếp cận hiện đại là biểu diễn mỗi sản phẩm bằng một **vector đặc trưng (embedding)** và tìm các vector gần nhất.

3.3.2. Giải pháp PostgreSQL

Extension **pgvector** bổ sung kiểu `vector(n)`, các toán tử khoảng cách (<-> Euclid, <=> cosine, <#> tích vô hướng) và hai loại index tìm láng giềng gần đúng: IVFFlat và **HNSW**. HNSW (Hierarchical Navigable Small World) xây đồ thị nhiều tầng, cho tốc độ truy vấn và độ chính xác cao, không cần huấn luyện trước như

IVFFlat. Đây là tính năng khiến PostgreSQL trở thành lựa chọn phổ biến cho ứng dụng AI/RAG trong hai năm gần đây [5].

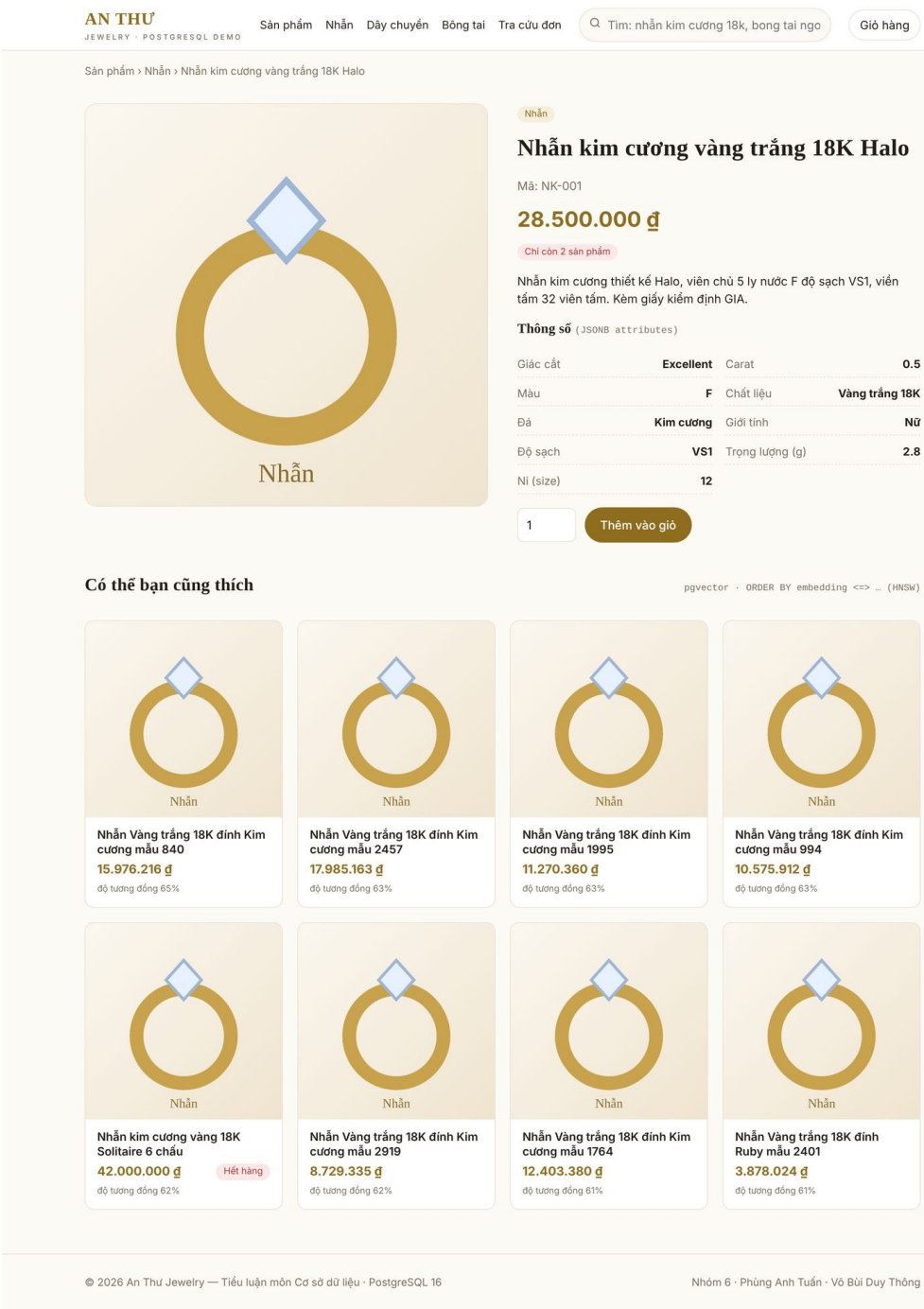
```
-- 002_core_tables.sql
embedding vector(256),
-- 004_indexes.sql
CREATE INDEX idx_products_embedding ON products USING hnsw (embedding
vector_cosine_ops);

-- api/src/routes/catalog.js: 8 sản phẩm gần nhất theo khoảng cách cosine
SELECT p.id, p.slug, p.name, p.price,
       round((1 - (p.embedding <=> me.embedding))::numeric, 3) AS similarity
FROM products p, (SELECT id, embedding FROM products WHERE slug = $1) me
WHERE p.id <> me.id AND p.status = 'active' AND p.embedding IS NOT NULL
ORDER BY p.embedding <=> me.embedding
LIMIT 8;
```

Về phía tạo embedding: trong môi trường sản xuất thường gọi mô hình AI (OpenAI text-embedding, CLIP cho ảnh…). Để tiểu luận chạy được không phụ thuộc dịch vụ ngoài, nhóm cài đặt embedding bằng kỹ thuật **feature hashing** (`api/src/embedding.js`): tách tên, mô tả, danh mục và từng cặp khoá=giá trị của JSONB thành token và bigram, băm mỗi token vào một trong 256 chiều với dấu ±, nhân trọng số (danh mục và thuộc tính ×4, tên ×3, mô tả ×1) rồi chuẩn hoá L2. Hai sản phẩm cùng chất liệu, cùng đá, cùng loại sẽ chia sẻ nhiều token nên vector gần nhau. Lược đồ và truy vấn không thay đổi nếu sau này thay bằng mô hình AI thật – chỉ cần đổi hàm `embedProduct()`.

3.3.3. Kết quả

Với "Nhẫn kim cương vàng trắng 18K Halo", 8 gợi ý đều là nhẫn vàng trắng 18K đính kim cương (độ tương đồng 61–65 %), trong đó có "Nhẫn kim cương vàng 18K Solitaire" (Hình 3.2). Truy vấn dùng `Index Scan using idx_products_embedding`, 0,23 ms so với 1,7 ms khi tính khoảng cách cho toàn bộ 3.033 vector (nhanh hơn ~7 lần). Khi thêm/sửa sản phẩm trên trang quản trị, API tính lại embedding và ghi trong cùng câu INSERT/UPDATE.



Hình 3.2: Trang chi tiết sản phẩm với khối "Có thể bạn cũng thích" từ pgvector

3.4. Table partitioning và BRIN index cho bảng đơn hàng

3.4.1. Bài toán

Bảng đơn hàng chỉ tăng, không giảm. Sau vài năm có thể hàng triệu dòng, trong khi nghiệp vụ hằng ngày chỉ chạm tới đơn gần đây: báo cáo "doanh thu tháng này", danh sách đơn chờ xử lý, lọc theo khoảng ngày. Một bảng khổng lồ làm index B-tree lớn, VACUUM chậm, lưu trữ dữ liệu cũ phải DELETE hàng loạt.

3.4.2. Giải pháp PostgreSQL

Declarative partitioning chia bảng thành các partition con theo RANGE/LIST/HASH ngay trong câu CREATE TABLE. Planner thực hiện **partition pruning**: chỉ quét partition có khả năng chứa dữ liệu. Tạo partition mới cho tháng tới chỉ là một CREATE TABLE … PARTITION OF; lưu trữ dữ liệu cũ là DETACH/DROP một partition, tức thời.

```
-- 003_orders_partitioned.sql
CREATE TABLE orders (
  id BIGSERIAL, customer_id BIGINT NOT NULL REFERENCES customers(id),
  status TEXT NOT NULL DEFAULT 'pending' CHECK (status IN (...)),
  subtotal NUMERIC(14,0) NOT NULL DEFAULT 0, shipping_fee NUMERIC(14,0) NOT
NULL DEFAULT 0,
  total NUMERIC(14,0) GENERATED ALWAYS AS (subtotal + shipping_fee) STORED,
  note TEXT, created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (id, created_at)            -- khoá chính phải chứa cột phân
vùng
) PARTITION BY RANGE (created_at);

-- Tạo sẵn 12 partition tháng của năm 2026 bằng vòng lặp PL/pgSQL
CREATE TABLE orders_2026_10 PARTITION OF orders
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
CREATE TABLE orders_default PARTITION OF orders DEFAULT;

-- Hàm tạo partition cho tháng bất kỳ (gọi từ cron hoặc admin)
SELECT ensure_orders_partition('2027-01-01');
```

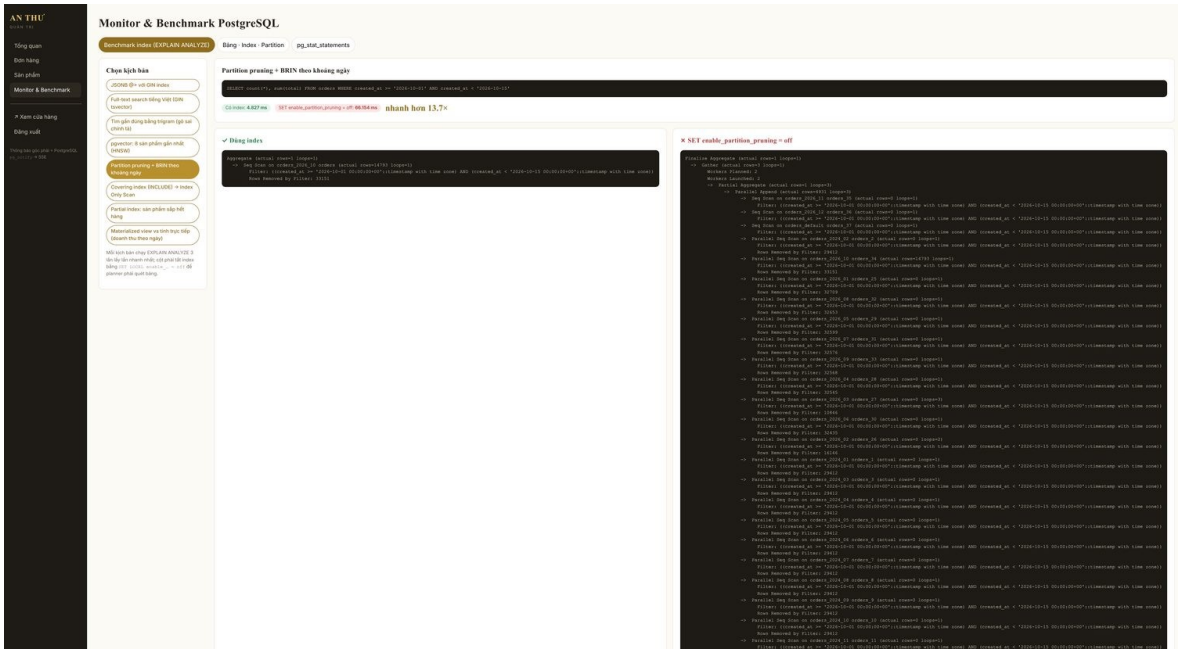
Trên cột thời gian của bảng ghi theo thứ tự, nhóm dùng **BRIN (Block Range Index)** thay vì B-tree. BRIN chỉ lưu giá trị min/max của mỗi khoảng 128 trang dữ liệu, nên kích thước gần như không đổi khi bảng lớn lên, trong khi vẫn loại bỏ được hầu hết khối không liên quan khi lọc theo khoảng ngày.

```
CREATE INDEX idx_orders_created_brin ON orders USING BRIN (created_at);

-- Truy vấn báo cáo chỉ quét partition tháng 10 (partition pruning)
SELECT count(*), sum(total) FROM orders
WHERE created_at >= '2026-10-01' AND created_at < '2026-10-15';
```

3.4.3. Kết quả

Để kiểm chứng partition ở quy mô thật, nhóm sinh thêm **1.035.853 đơn hàng** (1.543.358 dòng chi tiết, 312 MB) rải đều 34 tháng từ 01/2024 bằng lệnh `npm run seed:big`, tạo thành 37 partition. Với truy vấn doanh thu 2 tuần, planner chỉ quét **1/37 partition** và mất **3,3 ms**; khi tắt pruning phải Append cả 37 partition, **66 ms** (nhanh hơn ~20 lần). Báo cáo một tháng theo trạng thái: 10 ms so với 62 ms. Tổng kích thước BRIN trên 37 partition là **888 kB** so với **23 MB** của B-tree tương đương (nhỏ hơn 26 lần); BRIN tăng theo số khối chứ không theo số dòng nên khoảng cách này càng lớn khi dữ liệu tăng. Một kết quả trung thực khác: với partition theo tháng, mỗi partition chỉ ~30.000 dòng nằm gọn trong bộ nhớ đệm nên ép dùng BRIN để lọc 3 ngày (2,7 ms) không nhanh hơn quét tuần tự partition đó (1,9 ms) — nghĩa là khi đã partition, giá trị của BRIN nằm ở kích thước, còn tốc độ đến từ pruning. Trang quản trị hiển thị số dòng và dung lượng từng partition (thẻ Bảng · Index · Partition).



Hình 3.3: Partition pruning: chỉ quét 1 partition (trái) so với Append toàn bộ partition (phải)

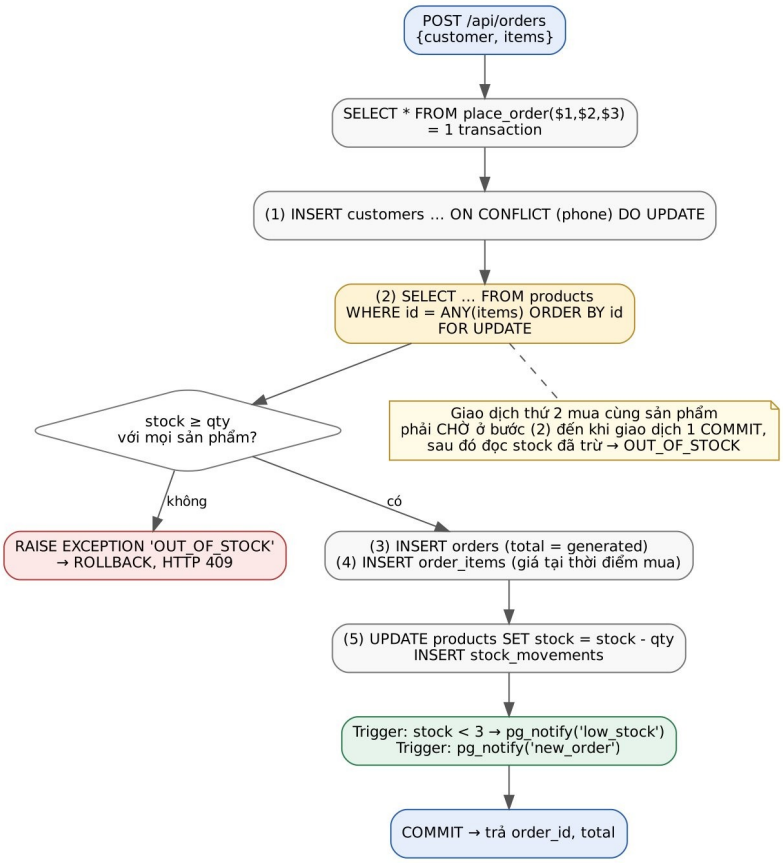
3.5. Transaction và SELECT … FOR UPDATE – chống bán quá tồn kho

3.5.1. Bài toán

Trang sức tồn kho ít: "Nhẫn kim cương vàng 18K Solitaire" chỉ còn 1 chiếc. Hai khách bấm "Đặt hàng" cùng lúc. Nếu ứng dụng chỉ `SELECT stock` rồi `UPDATE stock = stock - 1`, cả hai giao dịch cùng đọc được stock = 1, cùng cho là đủ, và cả hai cùng tạo đơn – cửa hàng bán 2 chiếc trong khi chỉ có 1. Đây là lỗi **race condition** kinh điển, không thể giải quyết bằng kiểm tra ở tầng ứng dụng.

3.5.2. Giải pháp PostgreSQL

Toàn bộ nghiệp vụ đặt hàng được gói trong **một hàm PL/pgSQL `place_order()`**, tức một giao dịch ACID: hoặc tất cả bước thành công, hoặc không có gì được ghi. Bên trong, câu `SELECT … FOR UPDATE` **khoá đúng những dòng sản phẩm** sắp mua: giao dịch thứ hai chạm tới cùng dòng phải chờ giao dịch thứ nhất COMMIT, sau đó đọc giá trị stock *đã bị trừ* và phát hiện hết hàng. Khoá được lấy theo thứ tự `ORDER BY id` để hai giao dịch mua nhiều sản phẩm luôn khoá cùng thứ tự, tránh deadlock. Ràng buộc `CHECK (stock >= 0)` trên bảng là hàng rào cuối cùng.



Hình 3.4: Luồng xử lý trong hàm place_order()

```
-- 005_order_transaction.sql (rút gọn)
CREATE OR REPLACE FUNCTION place_order(p_customer JSONB, p_items JSONB, p_note
TEXT DEFAULT NULL)
RETURNS TABLE (order_id BIGINT, order_total NUMERIC) LANGUAGE plpgsql AS $$
BEGIN
  -- (1) Upsert khách theo số điện thoại
  INSERT INTO customers (full_name, phone, email, address) VALUES (...)
  ON CONFLICT (phone) DO UPDATE SET full_name = EXCLUDED.full_name, ...
RETURNING id INTO v_customer_id;

  -- (2) Khoá các dòng sản phẩm (FOR UPDATE), theo thứ tự id để tránh
deadlock, rồi kiểm tra tồn
  FOR r IN
    SELECT p.id, p.name, p.price, p.stock, i.qty
    FROM jsonb_to_recordset(p_items) AS i(product_id BIGINT, qty INT)
    JOIN products p ON p.id = i.product_id
    WHERE p.status = 'active'
    ORDER BY p.id
    FOR UPDATE OF p
  LOOP
    IF r.stock < r.qty THEN
      RAISE EXCEPTION 'OUT_OF_STOCK: "%" chỉ còn % sản phẩm, bạn đặt %',
r.name, r.stock, r.qty;
    END IF;
    v_subtotal := v_subtotal + r.price * r.qty;
  END LOOP;

  -- (3)(4) Tạo đơn + chi tiết (lưu giá tại thời điểm mua)
```

```
  INSERT INTO orders (...) VALUES (...) RETURNING id INTO v_order_id;
  INSERT INTO order_items (...) SELECT ... FROM
jsonb_to_recordset(p_items) ...;

  -- (5) Trừ kho + nhật ký kho (trigger sẽ bắn NOTIFY nếu sắp hết)
  UPDATE products p SET stock = p.stock - i.qty FROM
jsonb_to_recordset(p_items) AS i(...) WHERE p.id = i.product_id;
  INSERT INTO stock_movements (product_id, delta, reason, ref_id) SELECT
i.product_id, -i.qty, 'order', v_order_id FROM ...;

  RETURN QUERY SELECT v_order_id, v_subtotal + v_shipping;
END $$;
```

Tầng API chỉ còn một câu `SELECT * FROM place_order($1, $2, $3);` thông điệp lỗi do PL/pgSQL RAISE được chuyển thẳng về giao diện với mã HTTP 409 để khách thấy "chỉ còn 1 sản phẩm".

3.5.3. Kết quả

Kịch bản kiểm chứng (`db/benchmark/oversell_test.sql`): đặt stock = 5 cho một sản phẩm, dùng pgbench cho **50 client đồng thời** mỗi client mua 1 chiếc. Kết quả: đúng **5 đơn thành công**, 45 giao dịch nhận lỗi OUT_OF_STOCK, stock cuối cùng = 0, không có giao dịch nào làm kho âm. Thông lượng với 30 client đặt hàng liên tục trên CSDL 1 triệu đơn đạt ~1.090 đơn/giây, độ trễ trung bình 28 ms. Huỷ đơn (`cancel_order()`) cũng là một giao dịch: trả kho, ghi nhật ký, đổi trạng thái – nếu bất kỳ bước nào lỗi thì không bước nào có hiệu lực.

3.6. Materialized view và LISTEN/NOTIFY cho dashboard quản trị

3.6.1. Bài toán

Dashboard cần doanh thu theo ngày trong 30 ngày, top sản phẩm bán chạy, doanh thu theo danh mục theo tháng. Mỗi lần mở trang mà JOIN và GROUP BY hàng triệu dòng `order_items` là lãng phí vì số liệu lịch sử không đổi. Ngoài ra người quản lý muốn được báo **ngay** khi một mã sắp hết hàng hoặc có đơn mới, không phải bấm tải lại trang; cách thông thường là polling định kỳ hoặc thêm Redis/RabbitMQ.

3.6.2. Giải pháp PostgreSQL

Materialized view lưu kết quả truy vấn thành bảng vật lý, có thể đánh index và chỉ tính lại khi gọi REFRESH. Với `REFRESH MATERIALIZED VIEW CONCURRENTLY` (yêu cầu một UNIQUE INDEX), việc làm mới không chặn người

đang đọc. Nhóm tạo 3 materialized view và một hàm `refresh_sales_reports()`; trang quản trị có nút "Refresh báo cáo", khi triển khai thật có thể gọi theo lịch (pg_cron hoặc cron hệ điều hành). Số liệu "hôm nay" vẫn tính trực tiếp vì chỉ chạm partition tháng hiện tại.

```
-- 006_reports_matview.sql
CREATE MATERIALIZED VIEW mv_sales_daily AS
SELECT date_trunc('day', o.created_at)::date AS day, count(*) AS orders,
       sum(o.total) AS revenue, sum(oi.qty) AS items_sold
FROM orders o JOIN order_items oi ON oi.order_id = o.id AND
oi.order_created_at = o.created_at
WHERE o.status <> 'cancelled' GROUP BY 1 WITH DATA;
CREATE UNIQUE INDEX ON mv_sales_daily (day);     -- cần cho REFRESH ...
CONCURRENTLY

CREATE OR REPLACE FUNCTION refresh_sales_reports() RETURNS VOID LANGUAGE
plpgsql AS $$
BEGIN
  REFRESH MATERIALIZED VIEW CONCURRENTLY mv_sales_daily;
  REFRESH MATERIALIZED VIEW CONCURRENTLY mv_top_products;
  REFRESH MATERIALIZED VIEW CONCURRENTLY mv_sales_by_category_month;
END $$;
```

LISTEN/NOTIFY là cơ chế pub/sub tích hợp sẵn: một phiên gọi `LISTEN kênh`, phiên khác (hoặc trigger) gọi `pg_notify(kênh, payload)` và mọi phiên đang lắng nghe nhận được thông điệp ngay khi giao dịch commit. Nhóm viết trigger trên `products` bắn kênh `low_stock` khi stock giảm xuống dưới 3 và trigger trên `orders` bắn kênh `new_order`. Backend giữ một kết nối riêng để LISTEN và chuyển tiếp xuống trình duyệt quản trị bằng Server-Sent Events.

```
-- 005_order_transaction.sql
CREATE OR REPLACE FUNCTION notify_low_stock() RETURNS trigger LANGUAGE plpgsql
AS $$
BEGIN
  IF NEW.stock < 3 AND NEW.stock < OLD.stock THEN
    PERFORM pg_notify('low_stock', json_build_object(
      'product_id', NEW.id, 'sku', NEW.sku, 'name', NEW.name, 'stock',
NEW.stock, 'at', now())::text);
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER trg_products_low_stock AFTER UPDATE OF stock ON products
FOR EACH ROW EXECUTE FUNCTION notify_low_stock();

// api/src/routes/events.js
await client.query('LISTEN low_stock'); await client.query('LISTEN
new_order');
client.on('notification', (msg) => {
  for (const res of subscribers) res.write(`event: ${msg.channel}\ndata: $
{msg.payload}\n\n`);
});
```

3.6.3. Kết quả

Trên CSDL 1 triệu đơn, đọc doanh thu 30 ngày từ materialized view mất **0,018 ms** so với **19,4 ms** khi tính trực tiếp trên bảng orders (nhanh hơn ~1.000 lần); top 10 sản phẩm bán chạy đọc từ `mv_top_products` mất 0,02 ms trong khi JOIN và GROUP BY 1,5 triệu dòng chi tiết mất **1,14 giây** – đây là loại truy vấn không thể chạy mỗi lần mở dashboard. REFRESH CONCURRENTLY cả 3 view mất vài giây và có thể đặt lịch. Khi tồn kho một sản phẩm giảm xuống 1, trình duyệt quản trị nhận thông báo trong dưới 100 ms mà không cần tải lại (Hình 3.5); khi khách đặt đơn mới, số liệu "hôm nay" trên dashboard tự cập nhật.



Hình 3.5: Thông báo "Sắp hết hàng" đẩy từ PostgreSQL (pg_notify) sang trình duyệt quản trị

3.7. Hệ thống chỉ mục và các loại index đặc biệt

Ngoài GIN, HNSW, BRIN đã trình bày, hệ thống dùng thêm bốn kỹ thuật index/cột mà PostgreSQL hỗ trợ tốt, mỗi index gắn với một truy vấn cụ thể của ứng dụng:

Bảng 3.1: Các index và cột sinh tự động khác trong hệ thống

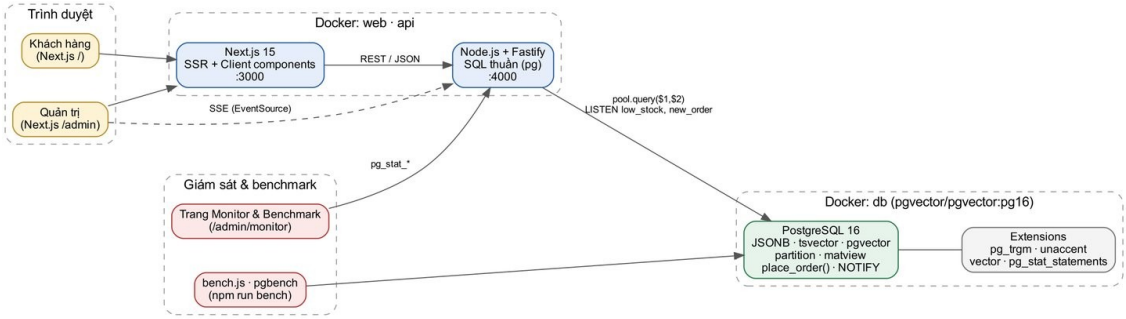
Loại	Index trong hệ thống	Truy vấn phục vụ – lợi ích
Partial index	`idx_products_low_stock ON products (stock, id) WHERE stock < 3 AND status = 'active'`	Dashboard "sắp hết hàng". Chỉ index các dòng thoả điều kiện → 40 kB so với 168 kB của khoá chính; 0,02 ms so với 1,7 ms quét bảng (86×).

Loại	Index trong hệ thống	Truy vấn phục vụ – lợi ích
Partial index	`idx_orders_pending ON orders (created_at DESC) WHERE status IN ('pending','confirmed')`	Danh sách đơn chờ xử lý trên 1 triệu đơn: 0,2 ms so với 98 ms quét bảng (~480×); index 17 MB vì đơn đã hoàn tất không chiếm chỗ.
Covering index (INCLUDE)	`idx_orders_customer_cover ON orders (customer_id, created_at DESC) INCLUDE (id, status, total)`	Lịch sử mua của khách: mọi cột cần lấy nằm trong index → *Index Only Scan*, Heap Fetches = 0; 0,2 ms so với 52 ms quét 1 triệu đơn (~250×).
Expression index	`idx_customers_email_lower ON customers (lower(email))`	Tìm khách theo email không phân biệt hoa thường mà vẫn dùng index.
B-tree kết hợp + partial	`idx_products_category_price ON products (category_id, price) WHERE status = 'active'`	Trang danh mục sắp xếp theo giá.
Generated column	`orders.total`, `order_items.line_total`, `products.search_vec`	Dữ liệu dẫn xuất do CSDL tính, luôn nhất quán, có thể index.

CHƯƠNG 4: XÂY DỰNG ỨNG DỤNG

4.1. Kiến trúc tổng thể

Hệ thống theo mô hình ba tầng: trình duyệt ↔ API ↔ CSDL, đóng gói thành 3 container Docker (db, api, web). Logic nghiệp vụ quan trọng nhất (đặt hàng, huỷ đơn, thông báo) nằm trong CSDL dưới dạng hàm và trigger; API chủ yếu nhận yêu cầu, gọi SQL và trả JSON.



Hình 4.1: Kiến trúc triển khai của hệ thống

Bảng 4.1: Công nghệ sử dụng

Thành phần	Công nghệ	Lý do chọn
CSDL	PostgreSQL 16 (image pgvector/pgvector:pg16)	Có sẵn extension vector; bật pg_stat_statements qua tham số khởi động
Backend	Node.js 22, Fastify 5, thư viện pg	Nhẹ, nhanh; `pg` cho phép viết SQL thuần với tham số $1, $2 – đúng yêu cầu không dùng ORM
Frontend	Next.js 15 (App Router), React 19	SSR giúp trang sản phẩm tải nhanh, một dự án cho cả trang khách và trang quản trị
Xác thực	JWT + bcrypt	Đơn giản, không cần phiên trên server
Giám sát & đo	pg_stat_statements, pg_stat_user_*; trang Monitor trong admin;	Mọi số liệu lấy thẳng từ view thống kê của PostgreSQL, không cần công cụ ngoài

Thành phần	Công nghệ	Lý do chọn
	bench.js; pgbench	
Triển khai	Docker Compose	Một lệnh dựng toàn bộ; chạy được trên máy cá nhân hoặc VM cloud

4.2. Backend: SQL thuần với Node.js

4.2.1. Lớp truy cập dữ liệu

Toàn bộ truy cập CSDL đi qua module `db.js` với ba hàm: `query(sql, params)` trả về mảng dòng, `one()` trả về một dòng, `tx(fn)` chạy nhiều câu trong một giao dịch (BEGIN/COMMIT/ROLLBACK). Tham số luôn được truyền qua `$1, $2` nên không có nguy cơ SQL injection. Kiểu NUMERIC và BIGINT được chuyển về Number để frontend dùng trực tiếp.

```
// api/src/db.js
export const pool = new Pool({ connectionString: process.env.DATABASE_URL,
max: 10 });
export async function query(text, params = []) { const res = await
pool.query(text, params); return res.rows; }
export async function one(text, params = [])   { return (await query(text,
params))[0] ?? null; }
export async function tx(fn) {
  const client = await pool.connect();
  try { await client.query('BEGIN'); const r = await fn(client); await
client.query('COMMIT'); return r; }
  catch (e) { await client.query('ROLLBACK'); throw e; }
  finally { client.release(); }
}
```

4.2.2. Trình chạy migration

Thay vì dùng thư viện, nhóm tự viết `migrate.js` khoảng 60 dòng: đọc thư mục `db/migrations`, so với bảng `schema_migrations`, chạy từng file chưa áp dụng trong một transaction (BEGIN → chạy SQL → ghi tên file → COMMIT, lỗi thì ROLLBACK) – nhờ PostgreSQL hỗ trợ DDL trong transaction nên không bao giờ có schema "nửa vời". Lệnh `npm run reset` xoá schema và chạy lại migrate + seed + embed khi phát triển.

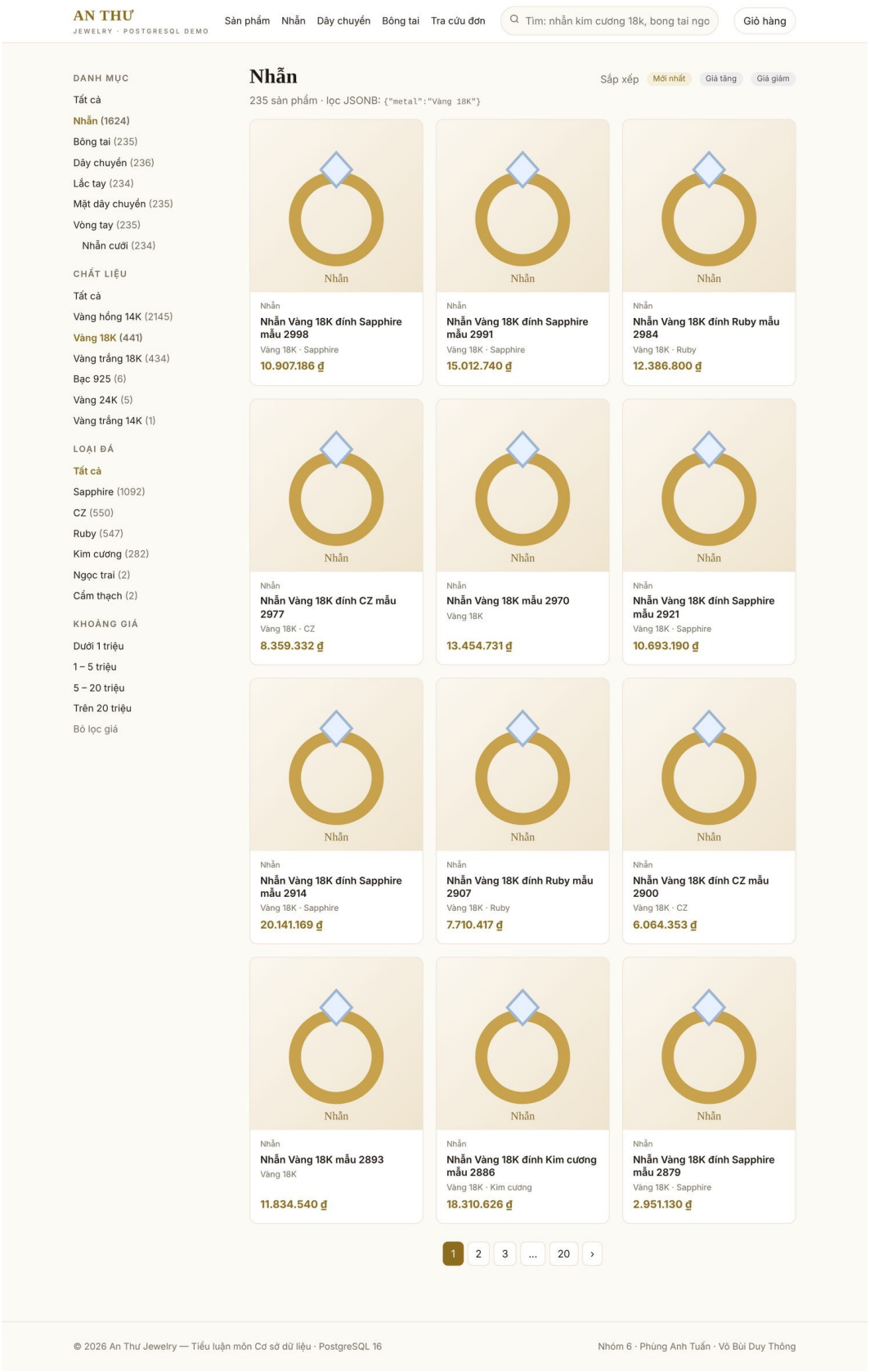
4.2.3. Các nhóm API

API chia thành sáu file theo nghiệp vụ: `catalog.js` (danh mục, danh sách lọc JSONB, chi tiết, gợi ý pgvector), `search.js` (full-text → trigram, gợi ý khi gõ), `orders.js` (đặt hàng qua `place_order()`, tra cứu đơn), `admin.js` (đăng nhập JWT, dashboard từ materialized view, CRUD sản phẩm có tính lại embedding, nhập kho, duyệt/huỷ đơn qua `cancel_order()`), `events.js` (LISTEN/NOTIFY → Server-Sent Events) và `monitor.js` (pg_stat_statements, thống kê bảng/index/partition, chạy EXPLAIN có/không index cho trang Benchmark).

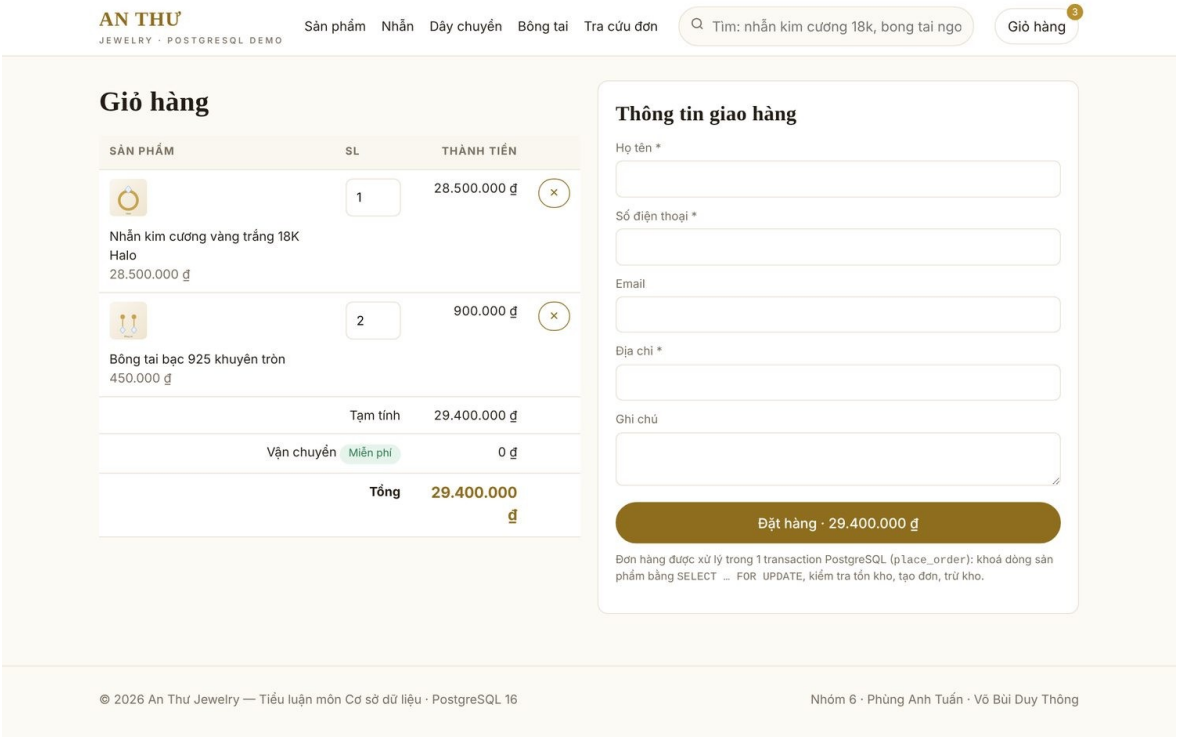
4.3. Frontend Next.js

Ứng dụng web gồm hai khu vực trong cùng một dự án Next.js. **Trang khách hàng** (`/`, `/san-pham`, `/san-pham/[slug]`, `/tim-kiem`, `/gio-hang`, `/tra-cuu`) dùng Server Components để gọi API và kết xuất HTML phía máy chủ; giỏ hàng lưu trong localStorage vì chưa cần ghi vào CSDL trước khi đặt. **Trang quản trị** (`/admin`, `/admin/san-pham`, `/admin/don-hang`, `/admin/monitor`) là Client Components, giữ JWT trong localStorage và mở một kết nối EventSource để nhận thông báo từ PostgreSQL.

4.4. Giao diện khách hàng

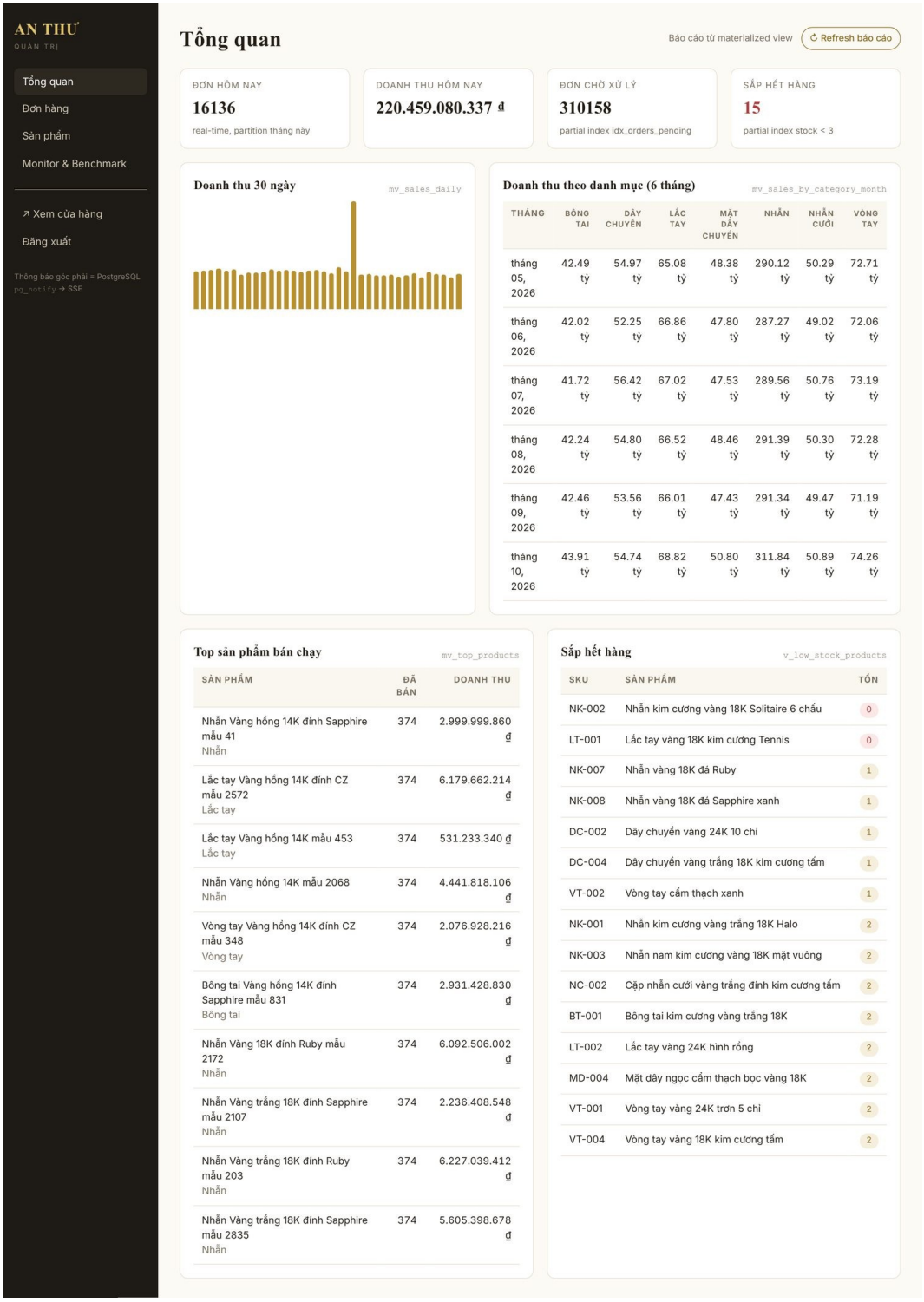


Hình 4.2: Danh sách sản phẩm lọc theo danh mục Nhẫn và chất liệu Vàng 18K (JSONB)

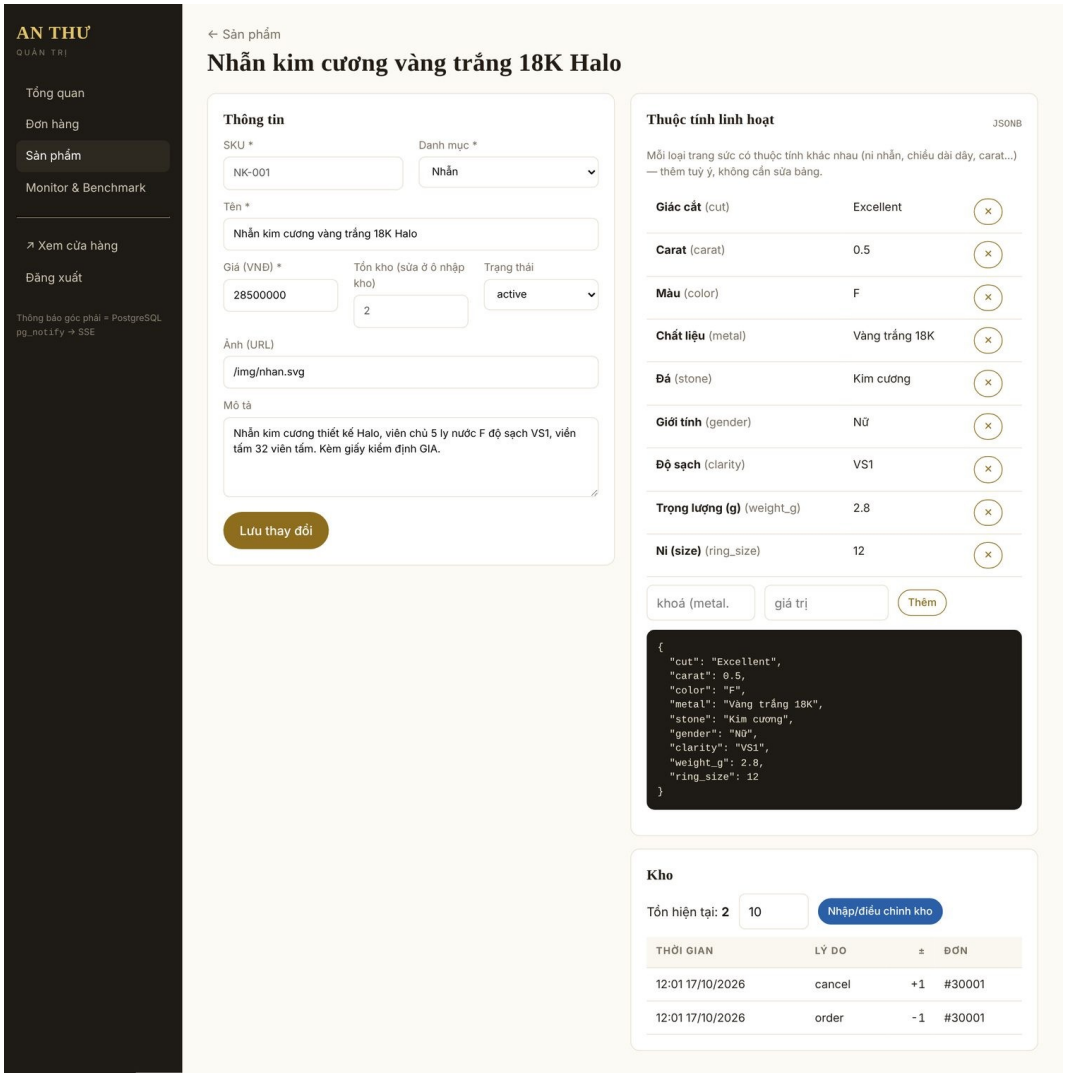


Hình 4.3: Giỏ hàng và biểu mẫu đặt hàng

4.5. Giao diện quản trị



Hình 4.4: Dashboard quản trị (dữ liệu từ materialized view)



Hình 4.5: Màn hình sửa sản phẩm – thêm/xoá thuộc tính JSONB và nhập kho

4.6. Triển khai bằng Docker Compose

File `docker-compose.yml` khai báo 3 dịch vụ: `db` (image chính thức pgvector/pgvector:pg16, bật pg_stat_statements), `api` (chờ db sẵn sàng rồi tự chạy migrate → seed → embed trước khi mở cổng) và `web` (Next.js standalone). Cổng 5432 được mở ra ngoài để kết nối DBeaver/psql trực tiếp vào CSDL.

```
docker compose up -d --build
# Web:     http://localhost:3000   (admin: /admin, tài khoản admin / admin123)
# API:     http://localhost:4000/api/health
# Monitor: http://localhost:3000/admin/monitor
```

Khi triển khai lên máy chủ thật chỉ cần đổi mật khẩu CSDL, `JWT_SECRET`, `ADMIN_PASSWORD` và gắn volume `pgdata` vào ổ đĩa bền.

CHƯƠNG 5: KIỂM THỬ, BENCHMARK VÀ GIÁM SÁT

5.1. Dữ liệu thử nghiệm: 1 triệu đơn hàng

Dữ liệu mẫu ban đầu (30.000 đơn) đủ để chạy ứng dụng nhưng quá nhỏ để thấy giá trị của partition và index – mọi bảng đều nằm gọn trong bộ nhớ đệm. Vì vậy nhóm viết thêm script `api/src/seed-big.js` (`npm run seed:big`) sinh **1.035.853 đơn hàng** và **1.543.358 dòng chi tiết** rải đều từ 01/2024 đến 10/2026, tự tạo partition cho từng tháng bằng hàm `ensure_orders_partition()`. Script chèn theo từng tháng bằng `generate_series` ngay trong CSDL (không đi qua ứng dụng) nên mất khoảng 100 giây; số lượng có thể đổi qua biến môi trường `BIG_ORDERS`.

Bảng 5.1: Môi trường và dữ liệu thử nghiệm

Chỉ số	Giá trị
PostgreSQL	16.15, container Linux 2 vCPU, cấu hình mặc định (shared_buffers 128 MB, work_mem 4 MB)
Sản phẩm	3.033 (40 viết tay + 2.993 sinh tự động), mỗi sản phẩm có JSONB, tsvector và vector 256 chiều
Đơn hàng	1.035.853 dòng, 37 partition theo tháng, 312 MB (kể cả index)
Chi tiết đơn	1.543.358 dòng, 241 MB
Khách hàng	1.006
Dung lượng CSDL	602 MB

5.2. Công cụ đo: bench.js

Để kết quả lặp lại được và không phụ thuộc thao tác tay, nhóm viết công cụ đo `api/src/bench.js` (`npm run bench`). Công cụ này:

- Chạy `VACUUM ANALYZE` rồi lần lượt 12 kịch bản, mỗi kịch bản **5 lần** bằng `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)` và lấy **trung vị** để loại nhiễu.

- Mỗi kịch bản chạy hai cấu hình: (A) bình thường – planner tự chọn index; (B) tắt loại scan tương ứng bằng `SET LOCAL enable_… = off` trong một transaction rồi ROLLBACK, hoặc so với câu truy vấn tương đương không dùng cấu trúc đặc biệt (materialized view).
- Đọc kế hoạch JSON để ghi lại loại scan, tên index, số partition thật sự bị quét, Heap Fetches và số buffer.
- Đo kích thước index: tạo tạm B-tree trên `orders(created_at)` để so với BRIN rồi xoá; partial, covering, khoá chính.
- Xuất `docs/benchmark/bench-report.md` (bảng Markdown) và `bench-report.json` (để vẽ biểu đồ), đồng thời in bảng ra màn hình.

```
$ cd api && npm run bench
Benchmark PostgreSQL — 5 lần/kịch bản, lấy trung vị
PostgreSQL 16.15 · 1,035,853 đơn / 37 partition / 312 MB · 1,543,358 chi tiết
· shared_buffers 128MB

Kịch bản                                    Có index   Không index  Nhanh hơn
Scan (có index)
JSONB @> với GIN (jsonb_path_ops)           0.062 ms   2.136 ms     34.5×
Bitmap Index Scan
Doanh thu 2 tuần: partition pruning + BRIN  3.276 ms   66.405 ms    20.3×
Seq Scan [1/37 partition]
Lịch sử mua của khách: covering (INCLUDE)   0.207 ms   51.885 ms    250.7×
Index Only Scan
Đơn chờ xử lý mới nhất: partial index       0.202 ms   97.83 ms     484.3×
Index Scan
Doanh thu 30 ngày: matview vs tính trực tiếp 0.018 ms  19.388 ms    1077×
Index Scan trên matview
...
Kích thước index: BRIN created_at 888 kB vs B-tree 23 MB (nhỏ hơn 26.3×)
```

5.3. Kết quả: có và không có index trên 1 triệu đơn hàng

Bảng 5.2: Thời gian thực thi có và không có index (trung vị 5 lần, 1.035.853 đơn hàng)

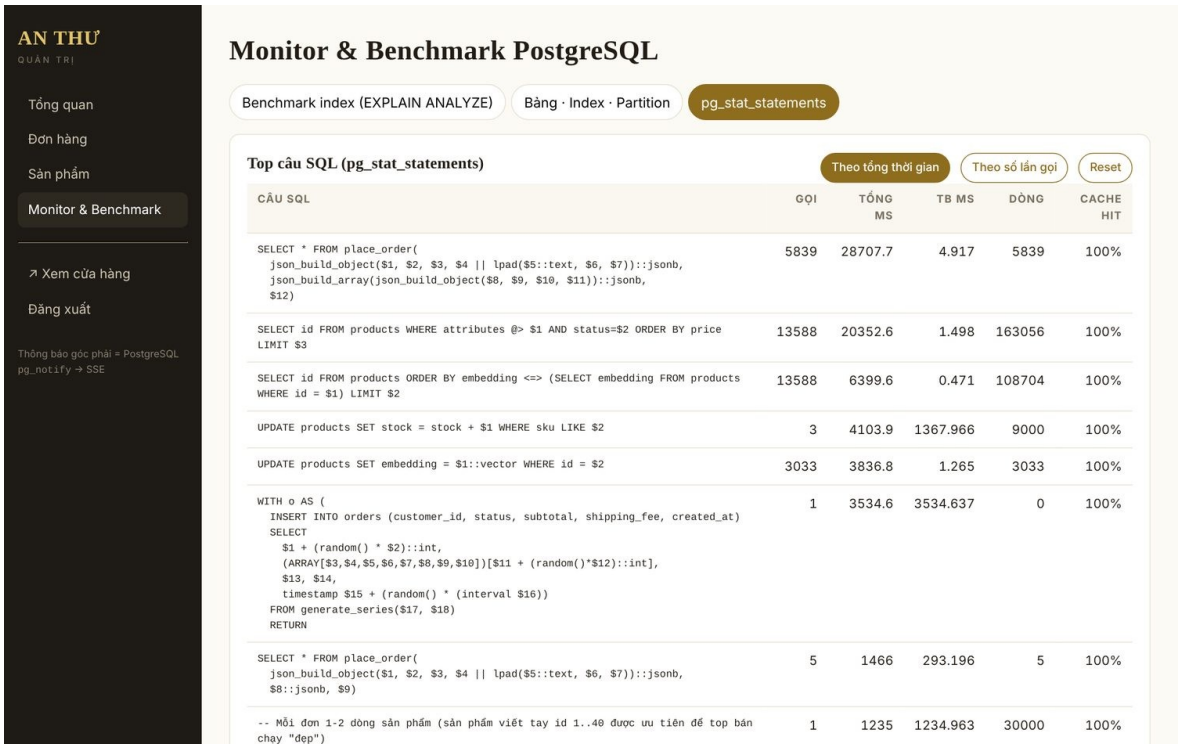
Kịch bản	Có index (ms)	Không index (ms)	Nhanh hơn	Scan khi có index
Lọc JSONB `@>` 2 thuộc tính	0,062	2,136	34×	Bitmap Index Scan (GIN)
Full-text search 3 từ	0,065	0,622	9,6×	Bitmap Index Scan (GIN)
Trigram gõ sai chính tả	6,35	16,3	2,6×	Bitmap Index

Kịch bản	Có index (ms)	Không index (ms)	Nhanh hơn	Scan khi có index
				Scan (GIN trgm)
pgvector 8 láng giềng	0,232	1,72	7,4×	Index Scan (HNSW)
Doanh thu 2 tuần (pruning)	3,28	66,4	20×	Seq Scan 1/37 partition
Doanh thu 1 tháng theo trạng thái	10,5	62,4	6×	Seq Scan 1/37 partition
Ép dùng BRIN lọc 3 ngày trong 1 partition	2,74	1,93	0,7×	Bitmap Index Scan (BRIN)
Lịch sử mua của khách (covering)	0,207	51,9	250×	Index Only Scan, Heap Fetches 0
Đơn chờ xử lý mới nhất (partial)	0,202	97,8	484×	Index Scan (partial)
Sắp hết hàng (partial)	0,020	1,71	86×	Bitmap Index Scan (partial)
Doanh thu 30 ngày (matview)	0,018	19,4	1.077×	Index Scan trên matview
Top 10 bán chạy (matview)	0,020	1.142	57.000×	Seq Scan trên matview

Bảng 5.3: Kích thước các index trên bảng orders và products

Index	Kích thước	Ghi chú
BRIN trên orders(created_at)	888 kB	Tổng 37 partition
B-tree trên orders(created_at)	23 MB	Tạo tạm để so sánh – BRIN nhỏ hơn 26 lần
Partial idx_orders_pending	17 MB	Chỉ chứa đơn pending/confirmed (~25 % số đơn)
Covering idx_orders_customer_cover	65 MB	INCLUDE (id, status, total) – đổi dung lượng lấy Index Only Scan
Partial idx_products_low_stock	40 kB	So với khoá chính products 168 kB

Nhận xét. (1) Lợi ích của index tăng theo kích thước dữ liệu: cùng truy vấn đơn chờ xử lý, trên 30.000 đơn index nhanh hơn ~16 lần, trên 1 triệu đơn nhanh hơn ~480 lần, vì quét tuần tự tăng tuyến tính còn index gần như không đổi. (2) Partition pruning là nguồn tăng tốc chính cho các báo cáo theo thời gian; khi đã partition theo tháng thì mỗi partition đủ nhỏ để BRIN không nhanh hơn quét tuần tự – giá trị của BRIN lúc này là kích thước (888 kB so với 23 MB) và nó vẫn hữu ích khi partition lớn hơn RAM. (3) Materialized view biến truy vấn hơn 1 giây thành micro-giây, đổi lại số liệu trễ đến lần REFRESH gần nhất – phù hợp cho báo cáo lịch sử, không dùng cho tồn kho. (4) Covering index tốn thêm 65 MB nhưng cho Index Only Scan (Heap Fetches = 0); với bảng lớn hơn RAM đây là khác biệt giữa đọc đĩa và không đọc đĩa.



Hình 5.1: Thống kê câu SQL từ pg_stat_statements trên trang quản trị

5.4. Kiểm chứng chống bán quá tồn kho

Dùng pgbench với kịch bản `oversell_test.sql`: đặt tồn kho của sản phẩm id = 2 bằng 5, khởi chạy 50 client đồng thời, mỗi client gọi `place_order()` mua 1 chiếc.

```
$ bash db/benchmark/run_bench.sh oversell
number of clients: 50
number of transactions actually processed: 5/50     ← chỉ 5 giao dịch thành
công
 id |                   name                    | stock
----+-------------------------------------------+-------
  2 | Nhẫn kim cương vàng 18K Solitaire 6 chấu |     0
```

Kết quả đúng như kỳ vọng: 5 đơn được tạo, 45 giao dịch còn lại bị từ chối với lỗi OUT_OF_STOCK, tồn kho về 0 và không bao giờ âm. Nếu bỏ `FOR UPDATE` và kiểm tra tồn kho ở tầng ứng dụng, số đơn thành công vượt quá 5 – minh hoạ vì sao phải để CSDL kiểm soát đồng thời.

5.5. Thông lượng với pgbench

Bảng 5.4: Kết quả pgbench (máy ảo 2 vCPU, cấu hình PostgreSQL mặc định)

Kịch bản (trên CSDL 1 triệu đơn)	Client	Thời gian	Giao dịch/giây	Độ trễ TB
Đặt hàng đồng thời (`place_order`, mỗi đơn 1 sản phẩm ngẫu nhiên)	30	10 s	≈ 1.090	28 ms
Tải đọc: full-text + lọc JSONB + gợi ý vector (3 câu/giao dịch)	20	10 s	≈ 850	24 ms

Khoảng một nghìn đơn mỗi giây trên CSDL đã có 1 triệu đơn là dư thừa so với nhu cầu của một cửa hàng trang sức, cho thấy một PostgreSQL đơn lẻ với index phù hợp đủ phục vụ mà không cần cache hay hệ thống tìm kiếm riêng.

5.6. Giám sát

pg_stat_statements ghi nhận mọi câu SQL đã chuẩn hoá: số lần gọi, tổng/trung bình thời gian, số dòng, tỉ lệ cache hit. Trang quản trị hiển thị top 25 câu theo tổng thời gian hoặc số lần gọi và có nút reset – câu nào trung bình chậm là ứng viên để thêm index. **pg_stat_user_tables / pg_stat_user_indexes / pg_inherits** cho biết số dòng, dung lượng, số lần quét tuần tự hay quét index của từng bảng, lượt dùng của từng index (index có `idx_scan = 0` lâu ngày nên xoá) và danh sách partition (thẻ Bảng ·

Index · Partition trong trang quản trị). Toàn bộ số liệu giám sát được lấy trực tiếp từ các view thống kê của PostgreSQL qua trang **Monitor & Benchmark** trong giao diện quản trị (ba thẻ: Benchmark EXPLAIN có/không index, Bảng – Index – Partition, pg_stat_statements), nên không cần cài thêm công cụ giám sát bên ngoài.

KẾT LUẬN

1. Kết quả đạt được

Tiểu luận đã xây dựng hoàn chỉnh một hệ thống bán hàng trang sức trên PostgreSQL 16 gồm lược đồ 8 bảng với đầy đủ ràng buộc, 6 file migration bằng SQL thuần, backend Node.js/Fastify không dùng ORM, frontend Next.js cho khách hàng và quản trị viên, đóng gói Docker Compose. Quan trọng hơn, mỗi tính năng nổi bật của PostgreSQL được gắn với một bài toán nghiệp vụ thật và được kiểm chứng bằng số liệu:

- **JSONB + GIN** giải quyết thuộc tính khác nhau giữa các loại trang sức, lọc nhanh hơn quét tuần tự ~30 lần.
- **Full-text search tiếng Việt** với unaccent và pg_trgm cho phép tìm không dấu, gõ sai chính tả, gợi ý khi gõ – không cần Elasticsearch.
- **pgvector + HNSW** mang tính năng gợi ý sản phẩm tương tự theo hướng AI vào ngay CSDL.
- **Partitioning + BRIN** kiểm chứng trên 1 triệu đơn: pruning chỉ quét 1/37 partition (nhanh hơn 20 lần), BRIN nhỏ hơn B-tree 26 lần.
- **Transaction + SELECT FOR UPDATE** trong hàm PL/pgSQL bảo đảm không bán quá tồn kho, kiểm chứng với 50 client đồng thời.
- **Materialized view + LISTEN/NOTIFY** cho dashboard nhanh gấp hàng nghìn lần trên 1 triệu đơn và cảnh báo thời gian thực mà không cần Redis hay hàng đợi.

Qua đó nhóm rút ra nhận định: PostgreSQL không chỉ là một CSDL quan hệ mà là một nền tảng dữ liệu đa mô hình; với một hệ thống thương mại điện tử quy mô vừa, việc tận dụng đúng các tính năng của PostgreSQL có thể thay thế nhiều thành phần hạ tầng riêng lẻ, giảm chi phí vận hành và độ phức tạp.

2. Hạn chế

- Embedding cho pgvector dùng kỹ thuật feature hashing nên chỉ phản ánh tương đồng về từ khoá/thuộc tính, chưa phải "hiểu" ngữ nghĩa hay hình ảnh như mô hình AI thật.
- Chưa có từ điển tiếng Việt cho full-text search (tách từ ghép, từ đồng nghĩa); hiện xử lý ở mức bỏ dấu và trigram.
- Mới đo đến 1 triệu đơn hàng trên một máy ảo 2 vCPU với cấu hình PostgreSQL mặc định; chưa tinh chỉnh tham số và chưa thử nghiệm replication/high availability.
- Nghiệp vụ còn đơn giản: chưa có thanh toán trực tuyến, khuyến mãi, nhiều kho.

3. Hướng phát triển

- Thay hàm embedding bằng mô hình đa phương thức (CLIP) để gợi ý theo ảnh sản phẩm; thử IVFFlat và tinh chỉnh tham số HNSW (m, ef_construction).
- Bổ sung từ điển tiếng Việt cho tsvector, kết hợp xếp hạng theo lượt bán.
- Tự động tạo partition và làm mới materialized view bằng pg_cron; lưu trữ partition cũ sang bảng nén hoặc S3.
- Thêm Row-Level Security để phân quyền nhiều nhân viên, streaming replication cho đọc/ghi tách biệt.
- Mở rộng nghiệp vụ: thanh toán, voucher, khách hàng thân thiết, nhiều chi nhánh với PostGIS.

TÀI LIỆU THAM KHẢO

1. DB-Engines, "DB-Engines Ranking – Trend of PostgreSQL popularity", https://db-engines.com/en/ranking, truy cập tháng 10/2026.
2. Redgate Software, "DB-Engines: PostgreSQL leads H1 2026 database growth as data platforms gain momentum", thông cáo báo chí 10/06/2026.
3. PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, https://www.postgresql.org/docs/16/ – các chương: JSON Types, Full Text Search, Table Partitioning, Index Types, Materialized Views, Explicit Locking, NOTIFY.
4. Stack Overflow, *Developer Survey 2024–2025*, mục Databases, https://survey.stackoverflow.co/.
5. Andrew Kane, *pgvector – Open-source vector similarity search for Postgres*, https://github.com/pgvector/pgvector.
6. PostgreSQL Global Development Group, *pg_trgm, unaccent, pg_stat_statements – Additional Supplied Modules*, https://www.postgresql.org/docs/16/contrib.html.
7. Fastify, *Fastify Documentation*, https://fastify.dev/; node-postgres, https://node-postgres.com/.
8. Vercel, *Next.js 15 Documentation*, https://nextjs.org/docs.
9. Đỗ Phúc, *Giáo trình Cơ sở dữ liệu*, NXB Đại học Quốc gia TP.HCM (tài liệu môn học).

PHỤ LỤC

Phụ lục A. Cài đặt và chạy trên máy cá nhân

Yêu cầu duy nhất: Docker Desktop (Mac/Windows) hoặc Docker Engine (Linux).

```
git clone https://github.com/phunganhtuan123/master-csdl.git
cd master-csdl
docker compose up -d --build       # lần đầu 2-3 phút: build image, migrate,
seed, embed
docker compose logs -f api         # chờ dòng "LISTEN low_stock, new_order" là
xong

# Web khách hàng : http://localhost:3000
# Trang quản trị : http://localhost:3000/admin   (admin / admin123)
# PostgreSQL     : localhost:5432  db jewelry  user postgres  pass postgres

docker compose exec api npm run seed:big   # (tuỳ chọn) thêm 1 triệu đơn hàng,
~2 phút
docker compose exec api npm run bench      # chạy công cụ đo, in bảng kết quả
docker compose down -v                     # xoá sạch để làm lại từ đầu
```

Phụ lục B. Kết nối DBeaver và các câu lệnh demo

Trong DBeaver chọn **Database → New Database Connection → PostgreSQL**, điền Host `localhost`, Port `5432`, Database `jewelry`, Username `postgres`, Password `postgres`, bấm Test Connection rồi Finish. Mở file `db/demo/demo_queries.sql` (File → Open), chọn từng khối lệnh và bấm Ctrl+Enter (Cmd+Enter). File gồm 8 mục đánh số trùng với chương 3: tổng quan dữ liệu; JSONB; full-text search; pgvector; partition; transaction chống bán quá kho (có hướng dẫn mở 2 tab SQL để thấy giao dịch thứ hai bị chờ); materialized view và NOTIFY; các index khác; giám sát. Một số lệnh tiêu biểu:

```
-- JSONB: lọc theo thuộc tính (GIN index)
SELECT sku, name, price FROM products
WHERE attributes @> '{"metal":"Vàng trắng 18K","stone":"Kim cương"}';

-- Full-text không dấu + trigram sai chính tả
SELECT name FROM products WHERE search_vec @@ plainto_tsquery('simple',
vi_unaccent('nhan kim cuong 18k'));
SELECT name FROM products WHERE vi_unaccent(name) % 'nhan kim cuog solitare';

-- pgvector: 8 sản phẩm giống NK-001
SELECT p.name, 1 - (p.embedding <=> me.embedding) AS similarity
FROM products p, (SELECT embedding FROM products WHERE sku='NK-001') me
ORDER BY p.embedding <=> me.embedding LIMIT 8;

-- Partition pruning: EXPLAIN chỉ thấy 1 partition
EXPLAIN (ANALYZE, COSTS OFF) SELECT count(*), sum(total) FROM orders
```

```
WHERE created_at >= '2026-09-01' AND created_at < '2026-09-15';

-- Đặt hàng: lần 2 báo OUT_OF_STOCK
UPDATE products SET stock = 1 WHERE sku = 'NK-002';
SELECT * FROM place_order('{"full_name":"A","phone":"0900000001"}',
'[{"product_id":2,"qty":1}]');
SELECT * FROM place_order('{"full_name":"B","phone":"0900000002"}',
'[{"product_id":2,"qty":1}]');

-- Materialized view vs tính trực tiếp
SELECT name, qty_sold FROM mv_top_products LIMIT 5;
SELECT refresh_sales_reports();
```

Phụ lục C. Cấu trúc mã nguồn

```
jewelry-shop-postgres/
├── docker-compose.yml      db + api + web
├── db/migrations/          001_extensions … 006_reports_matview (SQL thuần,
có chú thích)
├── db/seed/seed.sql        dữ liệu mẫu
├── db/demo/demo_queries.sql  câu lệnh demo cho DBeaver
├── db/benchmark/           pgbench: đặt hàng đồng thời, chống bán quá kho
├── api/src/                server.js, db.js, migrate.js, seed.js, seed-
big.js, embed.js, bench.js, routes/
├── web/app/(shop)/         trang khách hàng
├── web/app/admin/          trang quản trị
└── docs/                   tiểu luận, slide, benchmark/bench-report.md
```